# Nordic Tech Tour EMEA 2026: Hands-on Workshop

## 1. Welcome & Introduction

Welcome to our first Nordic Tech Tour EMEA hands-on workshop!

This year, alongside our usual Nordic product updates, we're introducing a second day dedicated entirely to hands-on work. Together, we'll build a project using Nordic products including the nRF54L15, nRF93M1, and nPM1300.

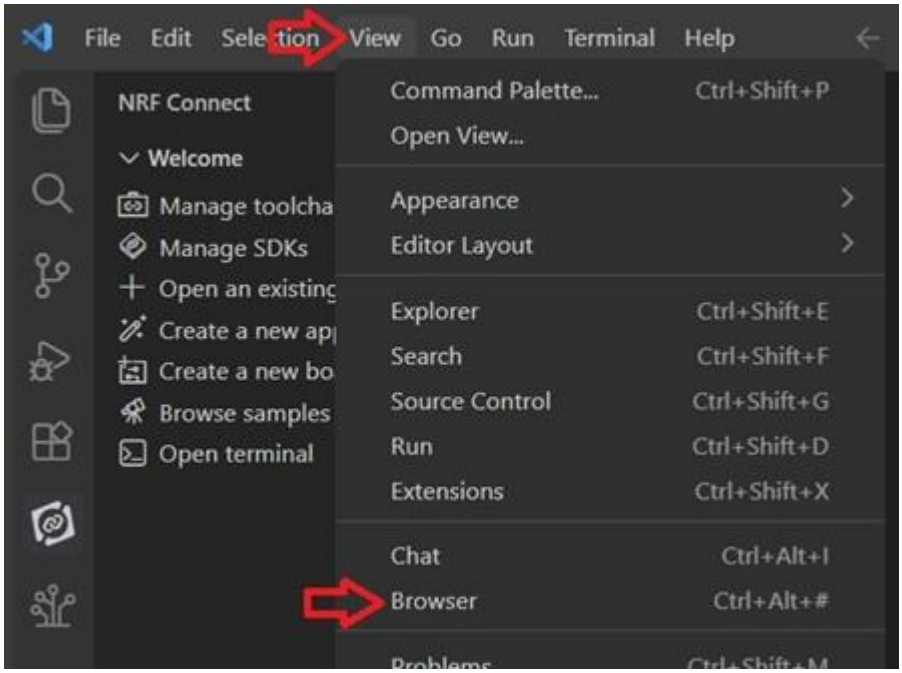
We're looking forward to a productive and enjoyable session together — welcome aboard!

### 1.1 How to use this guide

The Workshop Guide can be opened in any standard web browser or, if preferred, directly in VS Code using its integrated browser. This allows you to follow the hands-on, step-by-step instructions without leaving your development environment.

The following list outlines what is possible with this workshop guide:

- This guide walks you through the hands-on step by step.
- Use **Next step** to move forward, and **Previous** to go back.
- Chapters must be completed in order — a locked chapter ( 🔒 ) opens once every step in the chapters before it is checked off.
- Click a chapter's title in the sidebar to see a short summary of that chapter, if one is available.
- Your progress (steps completed / remaining) is shown at the top and updates automatically.
- Use the **Sidebar** switch (top row) to collapse the chapter list, and **Hide images** to hide pictures in the steps.
- Some steps have ⓘ **More info** buttons under their description — click one for extra background or troubleshooting tips.
- Click **Copy** on a code snippet to copy it to your clipboard.
- Open the ⚙ **Settings** menu (top right) to switch between dark and light mode, or to clear your saved progress.




*VS code built-in browser*

## 1.2 Enable Auto Save in VS Code

Throughout this workshop you'll be editing several files (source code, DeviceTree overlays, configuration files) and then triggering builds or flashing. To avoid confusion later — for example, forgetting to save before a build and wondering why your changes don't take effect — we recommend enabling **Auto Save** now, once, for the rest of the workshop.

**Steps:**

1. Open the **File** menu in the VS Code menu bar.
2. Click **Auto Save** to toggle it on. A checkmark next to the entry confirms it's enabled.

That's it — from now on, any change you make to a file is written to disk automatically after a short delay (or when you switch focus away from the editor), so you won't need to press **Ctrl+S** / **Cmd+S** throughout the workshop.

### But I want to save it manually

If you prefer to save manually and know what you're doing, you can skip this step — just remember to save (**Ctrl+S** / **Cmd+S**) before every build in the steps ahead, since an unsaved change will not be picked up by the build system.

## 1.3 Overview of Our Project

Today we will develop step-by-step an application that covers following functionalities:

- Sensor: first, we use a simultated sensor. And later a real sensor
- Bluetooth: we will use Bluetooth to send battery level to a smartphone. Code will be created with AI-assisted development.
- PMIC: configure device for voltage supply and use fuel gauge
- Cellular: Get location information and send sensor data to cloud.

HANDS-ON WORKSHOP

# Workshop Solution – Step by Step

We will build a complete application by adding the following components step by step.

## 1 Sensor



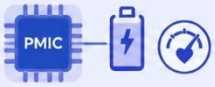

Start with a simulated sensor to collect data.
At the end, replace it with a real sensor.

## 2 Bluetooth



Use Bluetooth to send a notification of the remaining battery charge.

## 3 PMIC




Use the PMIC for power supply and to measure battery parameters (voltage, current, temperature).

Report state of charge and state of health using the Nordic fuel gauge algorithm.

## 4 Cellular



Use cellular to get location information and upload sensor data to the cloud.

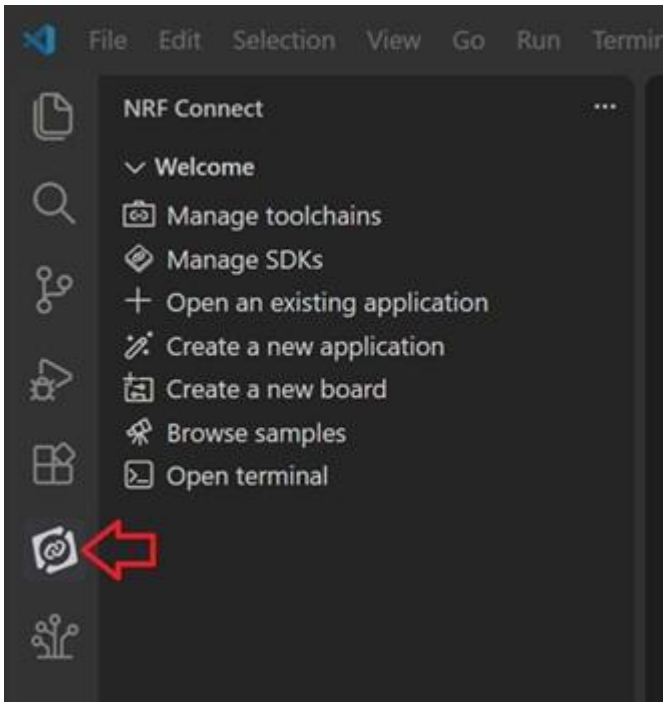


# 2. 1. Copy hello_word Sample

By the end of this chapter, you will have created your first application from a Zephyr/nRF Connect SDK sample, built it for the nRF93M1 DK, flashed it onto your kit, and confirmed it's running by reading its output over a serial connection. This chapter also introduces the core nRF Connect for VS Code workflow (create → configure → build → flash) that you'll reuse for every later chapter.

## 2.1 Open the nRF Connect Extension

Click the **nRF Connect** icon in the VS Code left sidebar (the Nordic "N" logo). This opens the extension's main panel, where you manage applications, build configurations, and connected devices.

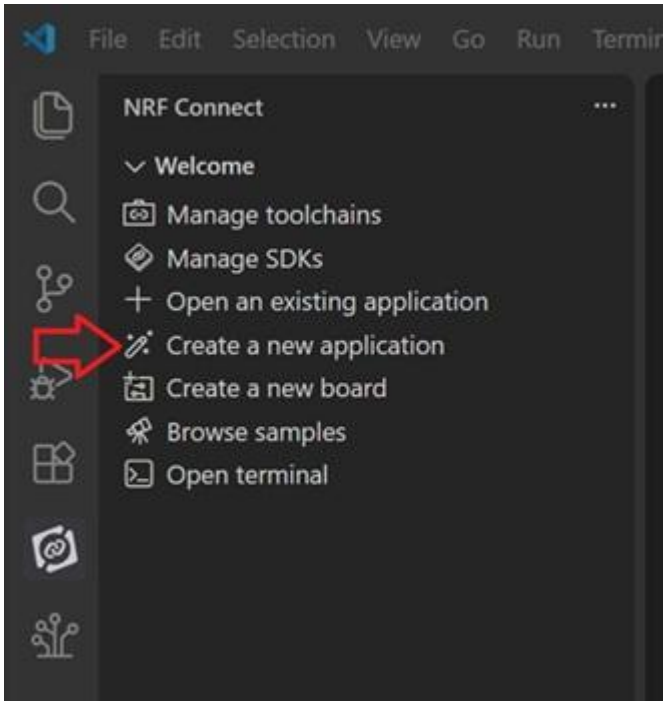



*nRF Connect*

## 2.2 Create a New Application

In the nRF Connect panel, click **Create New Application**.

An application in this context is just a folder containing your source code, build settings, and configuration files — everything Zephyr needs to compile firmware for your board.

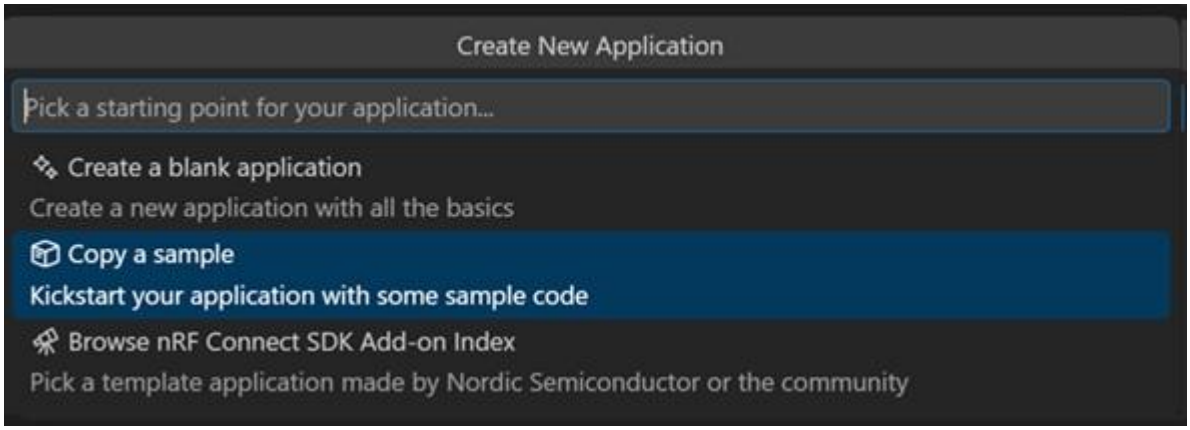


*Create new aplication*

## 2.3 Copy the hello_world Sample

Rather than writing an application from scratch, you'll start from one of Zephyr's built-in samples and copy it into your own workspace. This is the normal way to start a new nRF Connect SDK project.

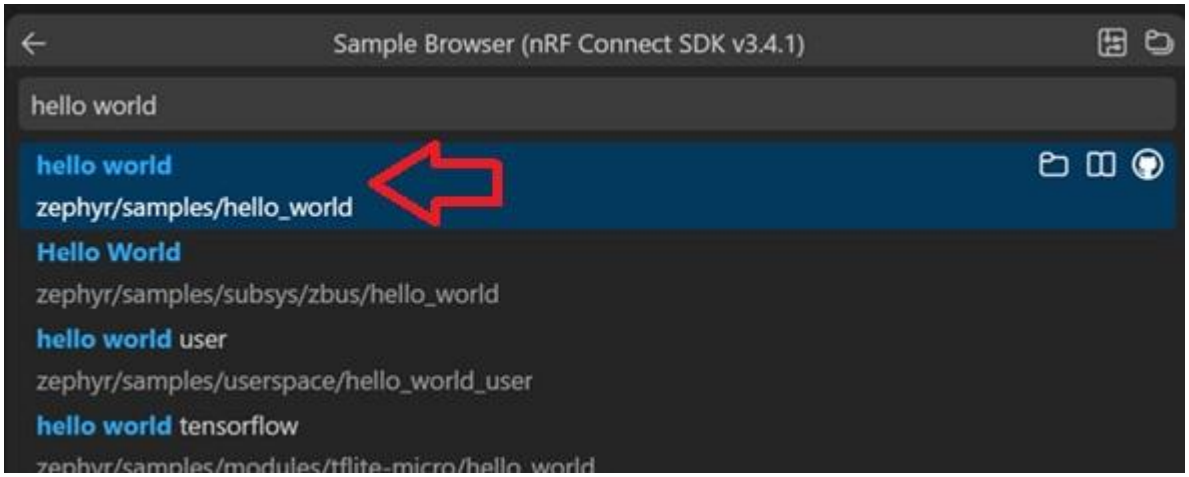
- Click **Copy Sample**.



*Copy Sample*

## 2.4 Select Zephyr's hello_world sample

In the search box, type `hello world`, then select the sample.
⚠️ **Take care:** several samples can match this search. Make sure you pick the one with the path `zephyr/samples/hello_world` — this is Zephyr's own minimal example, not a vendor-specific variant.



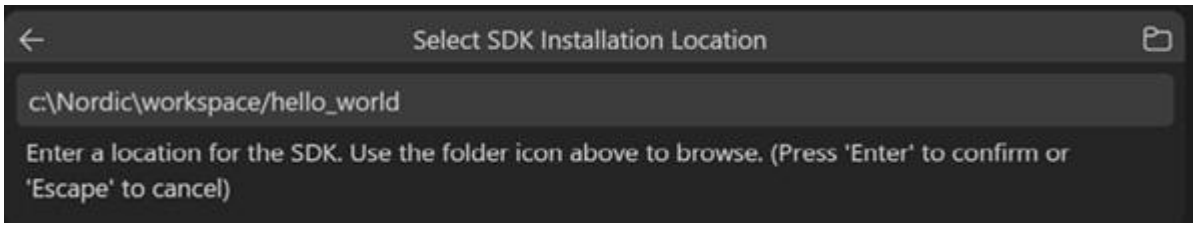
*Zephyr hello_world sample*

## 2.5 Save the Project in your own Workspace Directory

Choose a destination folder and confirm. VS Code will copy the sample files into your new application folder.

⚠️ **Keep your workspace path short. Recommendation:** place your application in a short root path, for example `C:\nordic\workspace\hello_world`

Once you've confirmed the path by pressing Enter, you can choose whether to open a new VS Code instance (“Open in New Window”) or to display the project in the current instance (“Open”).



*Choose application location*

### Why to keep your workspace path short?

Windows has a hard 260-character limit on full file paths. The build process for this SDK generates several nested subfolders (build files, CMake cache, generated headers, per-core output, and so on), and their combined path can get long fast. If your workspace already starts somewhere deep — e.g. `C:\Users\YourName\Documents\Workshops\2026\NordicWorkshop\...` — the build can fail with confusing "permission denied" or "file not found" errors that have nothing to do with your code.

**Recommendation:** place your application in a short root path, for example `C:\nordic\workspace\hello_world`, rather than several folders deep inside your user profile or a synced cloud folder (OneDrive/Dropbox paths are usually long and best avoided entirely for this reason).

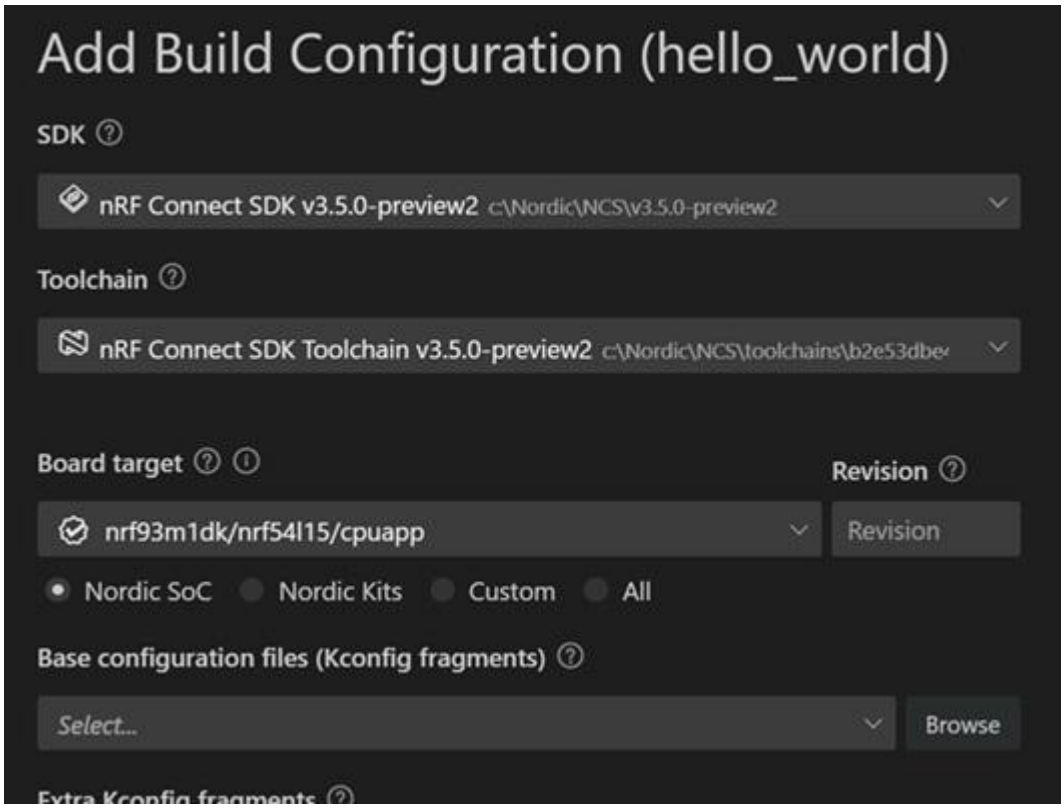
## 2.6 Add a Build Configuration

A build configuration tells the toolchain *which board* and *which core* to compile the firmware for. Without one, there's nothing to build.

1. Open the **nRF Connect** view.
2. In the **Application** section, click **Add Build Configuration**.

3. Under **Board target**, select: nrf93m1dk/nrf54l15/cpuapp
4. Leave the other options at their defaults.
5. Scroll down and click **Generate and Build**.

Wait for the build to finish — you'll see a "Build complete" message in the build log at the bottom of the window. If the build fails, check that the board target was typed/selected exactly as above.



*Add build configuration*

### Naming Convention for Board Targets

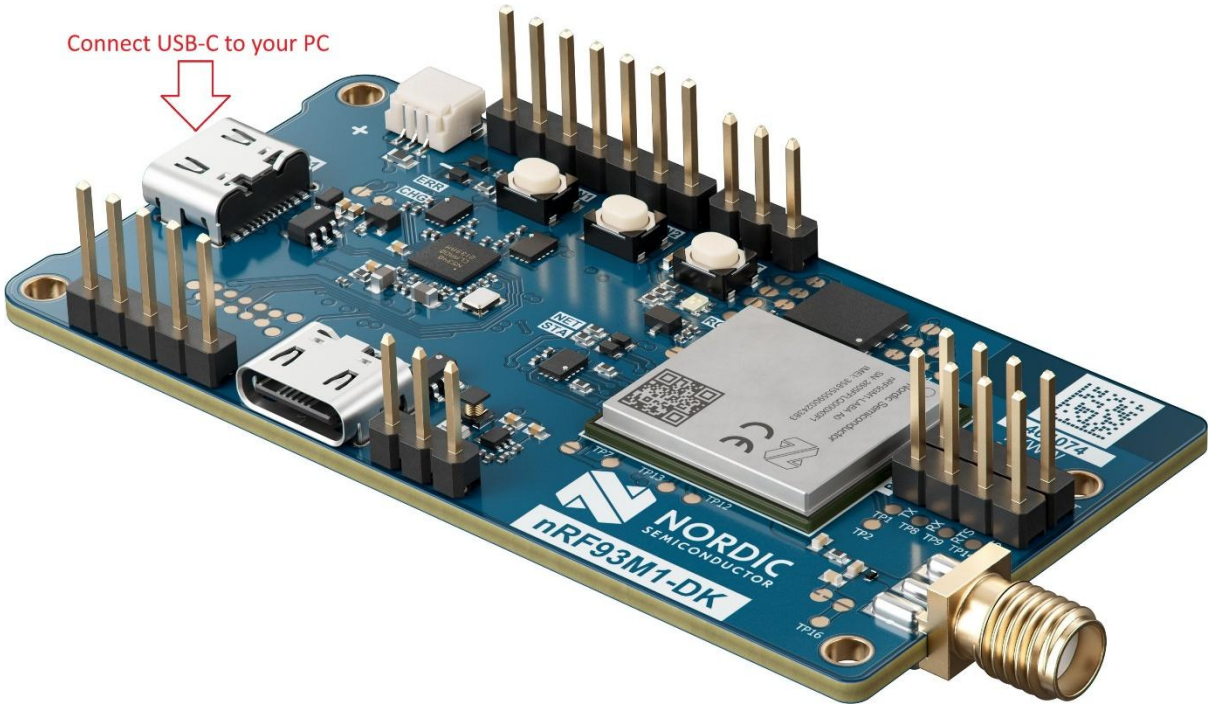
The `cpuapp` qualifier tells the build system which CPU core on the SoC to build for. The nRF54L15 is a multi-core chip: it has a main **application core** (`cpuapp`) that runs your everyday application code, and a small, low-power **FLPR core** (`cpuflpr`, a RISC-V co-processor) intended for offloading specific peripheral or real-time tasks. For this workshop, we always target `cpuapp` — it's the general-purpose core, has full support for logging, sensors, and connectivity, and is where you should build and run applications unless a task specifically calls for the FLPR core.

### What would “nrf54l15/cpuapp/ns” be used for?

Alongside plain `cpuapp`, the nRF54L15 also supports **`.../nrf54l15/cpuapp/ns`**. This builds your application to run in the **non-secure** execution environment, with **Trusted Firmware-M (TF-M)** occupying the secure side underneath it. TF-M isolates security-critical operations — cryptographic keys, secure storage, firmware update handling — from your application code, so a bug or vulnerability in your application can't reach into that protected area.

## 2.7 Prepare the development kit

Connect USB-C (USB1) to your PC. The antenna and SIM card aren't needed for this step.




*nRF93M1-DK*

## 2.8 Flash the Application

Now you'll load the compiled firmware onto the physical board.

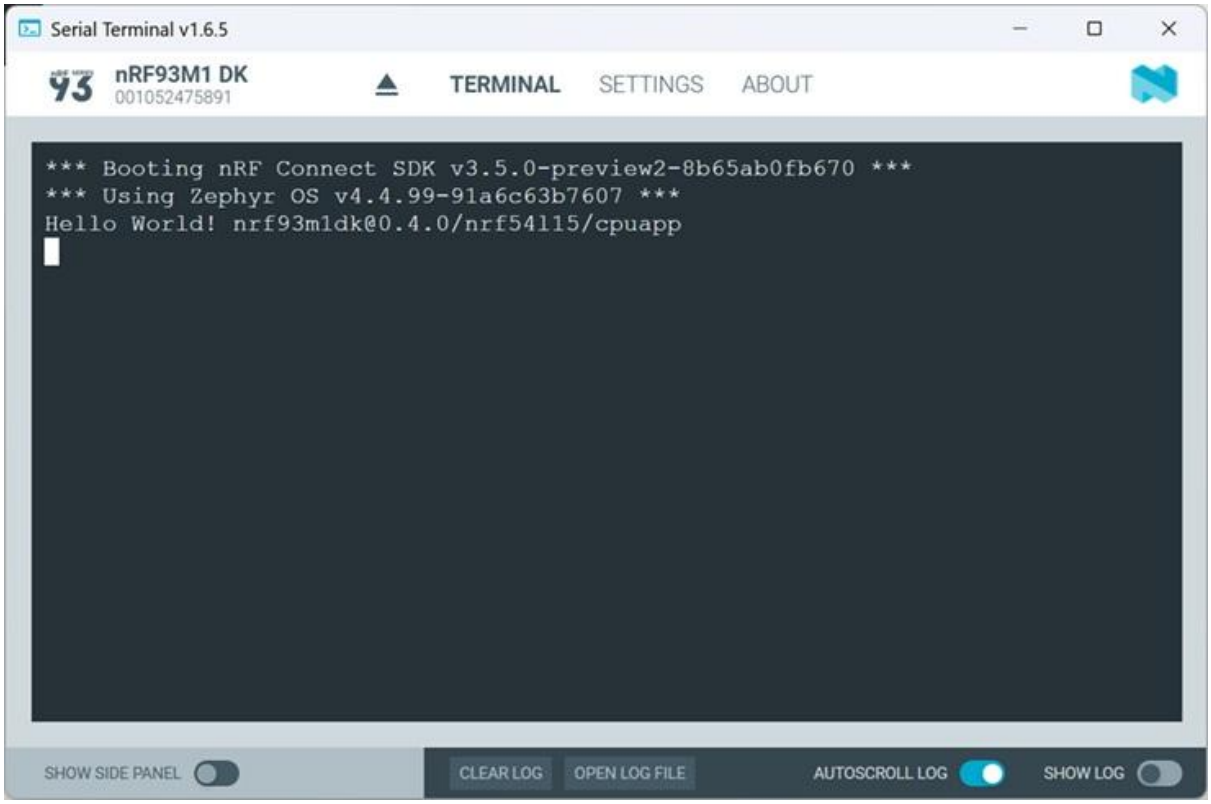
In the nRF Connect view, under **Actions**, click **Flash**.

## 2.9 Verify the Serial Terminal Output

We'll now check whether the compiled firmware runs on the board.

1. Open a **Serial Terminal** (via the nRF Connect **Connected Devices** panel or the **Serial Terminal** icon) and select your device's COM/serial port.
2. Press the **Reset** button on the DK, or power-cycle it, and watch the terminal.

You should see output similar to:




*Serial Terminal*

# 3. 2. Sensor - simulated sensor

**Learning Objectives**

By the end of this chapter you will be able to:

- Explain what Zephyr's sensor subsystem does and why applications use it instead of talking to hardware directly.
- Enable the sensor subsystem in a Zephyr/nRF Connect SDK project via Kconfig.
- Describe a virtual sensor device in the DeviceTree and bind it to your application.
- Read a sensor value in application code using the standard sensor API.
- Build, flash, and verify the result on your nRF93M1 DK.

**Background: Why a Sensor Subsystem?**

Every sensor — a real accelerometer, a temperature IC, or a simulated one — has its own register map and bus protocol. If application code talked to each sensor directly, swapping one sensor for another (or testing without hardware at all) would mean rewriting your application logic.

Zephyr solves this with a **common sensor API**: a single, standardized set of functions (`sensor_sample_fetch()`, `sensor_channel_get()`, …) that every sensor driver implements. Your application code calls these generic functions; the driver underneath handles the device-specific details. This is why we can develop and test with a **simulated sensor** now, and later swap in a real physical sensor with little or no change to `main.c`.

For this exercise we use nRF Connect SDK's built-in **sensor simulator** (`nordic,sensor-sim`), which generates plausible temperature, pressure, humidity, and acceleration values without any physical hardware.

## 3.1 Extending the Existing Project

Each chapter builds upon the project developed in the previous chapter. All concepts, instructions, and implementation steps introduced in the current chapter should be applied directly to the existing project.

## 3.2 Enable the sensor subsystem (Kconfig)

Open `prj.conf` in your project folder and add below KCONIFG settings.

`prj.conf`

```
# Enable Zephyr's common sensor API and sensor drivers
CONFIG_SENSOR=y

# Enable Floating Point output
CONFIG_CBPRINTF_FP_SUPPORT=y
```

### CONFIG_SENSOR

`CONFIG_SENSOR=y` pulls in the sensor subsystem core and enables the sensor drivers relevant to your board configuration — including the simulated sensor driver, which only needs a matching DeviceTree node to activate (next step).

### Why is CONFIG_CBPRINTF_FP_SUPPORT needed?

Zephyr's `printk()` uses its own lightweight formatting implementation, called **cbprintf**. Floating-point formatting is disabled by default because it requires additional code and increases the application's footprint.

When you enable CONFIG_CBPRINTF_FP_SUPPORT=y you tell Zephyr:

Include support for formatting floating-point values with `printk()`.

## 3.3 Describe the sensor in the DeviceTree

Zephyr drivers are matched to hardware (real or virtual) through the **DeviceTree**. Since the simulated sensor isn't part of the nRF93M1 DK's default board definition, we add it ourselves with a **DeviceTree overlay** — a file that layers extra nodes on top of the board's base DeviceTree without modifying the board files themselves.

Create a file named **`app.overlay`** in your project's root folder (the same folder as `prj.conf`) with the following DeviceTree content.

`app.overlay`

```
/ {
     simulated_sensor: sensor_sim{
          compatible = "nordic,sensor-sim";
          status = "okay";
          acc-signal = "wave";
          base-temperature = < 20 >;
          base-pressure = < 98 >;
     };
};
```

### What is a DeviceTree overlay?

The nRF Connect SDK (and Zephyr, which it's built on) describes hardware using **DeviceTree** — a text-based format that tells the firmware which peripherals exist, how they're wired, and how they should be configured. Every board has a base DeviceTree file; for the nRF93M1-DK's application core, this is `nrf93m1dk_nrf54l15_cpuapp.dts`.

Rather than editing that board file directly (which affects every project using this board), we use an **overlay file** — `app.overlay`. An overlay lets us add or override DeviceTree settings *just for our application*, without touching the board definition. This is the standard, recommended way to enable a sensor or peripheral that the board file doesn't already configure for you.

The nRF Connect SDK build system automatically picks up `app.overlay` if it's placed in the project root — no extra CMake configuration is needed.

### simulated_sensor

A **node label** — lets us refer to this node from C code without knowing its full DeviceTree path.

### compatible = "nordic,sensor-sim"

Tells Zephyr which driver to bind to this node — this string is what triggers the simulated sensor driver to build.

**status = "okay"**

| | |
|---|---|
| | Marks the node as active. A node with `status = "disabled"` is ignored, even if the driver is enabled. |

**acc-signal = "wave"**

| | |
|---|---|
| | Configures the simulated accelerometer to output a smooth waveform instead of a flat/random signal. |

**base-temperature = <20>**

| | |
|---|---|
| | Baseline value (°C) the simulated temperature channel fluctuates around. |

**base-pressure = <98>**

Baseline value (kPa) the simulated pressure channel fluctuates around.

## 3.4 Bind the sensor in application code

The sensor API is declared in `sensor.h`. Open `main.c` and add the includes and device handle.

`main.c`

```
#include <zephyr/kernel.h>
#include <zephyr/drivers/sensor.h>

#define SENSOR_NODE DT_NODELABEL(simulated_sensor)
const struct device *const dev = DEVICE_DT_GET(SENSOR_NODE);
```

**DT_NODELABEL(siumlated sensor)**

`DT_NODELABEL(simulated_sensor)` resolves, **at compile time**, to the DeviceTree node labeled `simulated_sensor:` in `app.overlay`.

**DEVICE_DT_GET(...)**

`DEVICE_DT_GET(...)` turns that DeviceTree node into a `const struct device *` handle — the same kind of handle used for *any* Zephyr or nRF Connect SDK device driver (sensors, UARTs, GPIOs, etc.).

At this point `dev` is a compile-time constant pointer to your sensor device — but drivers still need to initialize at boot, so we check that next.

## 3.5 Verify the sensor initialized correctly

Hardware (and simulated hardware) initializes during system boot. Before using a device, always confirm it's ready. Insert the following code at the beginning of `main()`, right before your main loop.

main.c

```
    if (!device_is_ready(dev)) {
        printk("Sensor is not ready %s\n", dev->name);
        return 0;
    }
    printk("Sensor is ready!\n");
```

### device_is_ready()

`device_is_ready()` checks that the driver's init function ran and completed without error. **Always guard sensor (and other driver) use with this check** — skipping it is a common source of confusing crashes when a DeviceTree node is misconfigured or disabled.

## 3.6 Read the sensor in a loop

Reading a Zephyr sensor is a **two-step process**, and it's important to understand why:

1. `sensor_sample_fetch(dev)` — talks to the device (real bus transaction, or in this case, the simulator's internal logic) and pulls a *fresh* reading into the driver's internal buffer.
2. `sensor_channel_get(dev, channel, &value)` — reads *one specific channel* out of that already-fetched buffer, without touching the device again.

Add the following code as the main loop.

main.c

```
    struct sensor_value temp;

    while (true) {
        // Fetch a sample from the sensor and store it in an internal driver buffer
        if (sensor_sample_fetch(dev) < 0) {
            printk("Error: Sensor sample update error\n");
            return 0;
        }

        // Get a reading from a sensor device (read temperature)
        if (sensor_channel_get(dev, SENSOR_CHAN_AMBIENT_TEMP, &temp) < 0) {
            printk("Error: Cannot read temperature channel\n");
            return 0;
        }

        /* display temperature */
        printk("Temperature: %.1f C\n", sensor_value_to_double(&temp));

        k_sleep(K_MSEC(2000));
    }
```

### Why is it split?

This split exists because many sensors report several channels (temperature, pressure, humidity, acceleration on 3 axes) from a *single* bus transaction. Fetching once and then getting multiple channels avoids unnecessary, repeated hardware access.

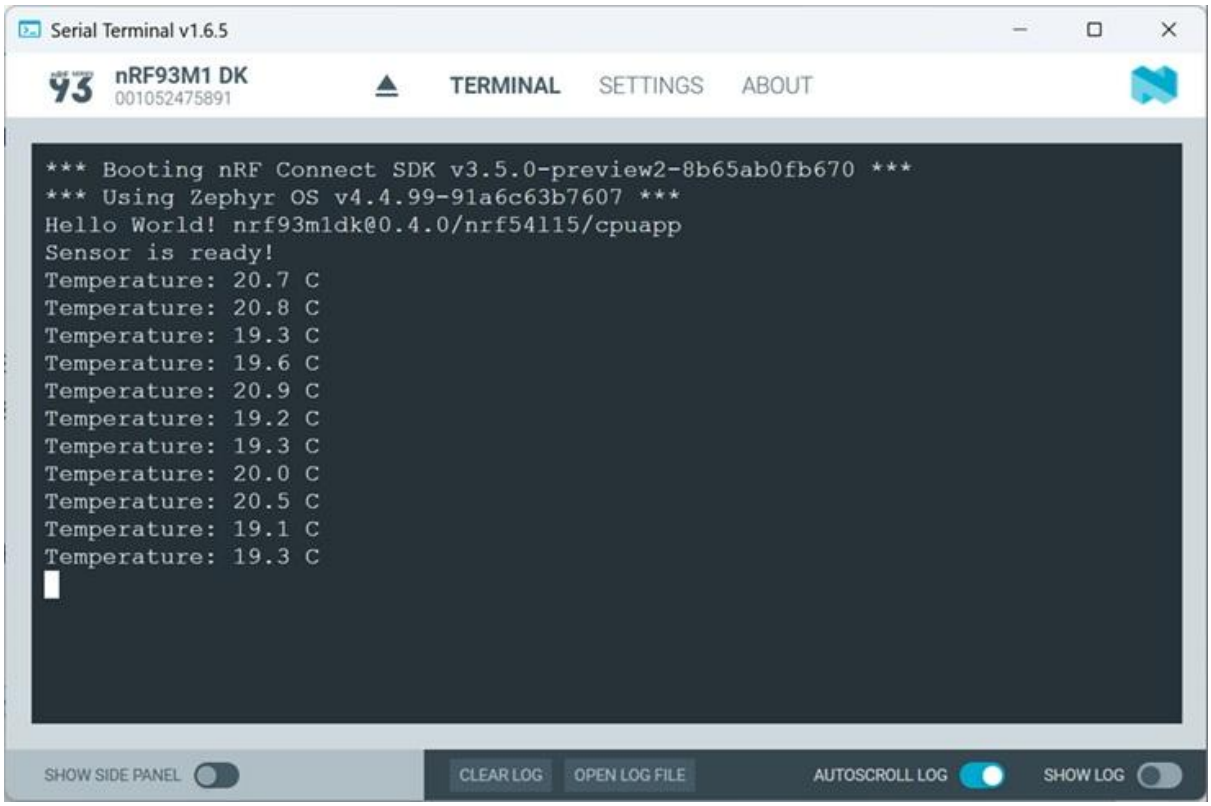
### sensor_value_to_double()

Values come back as a `struct sensor_value` — a fixed-point type (`val1` = integer part, `val2` = fractional part in millionths) chosen so the API works even without a hardware FPU. `sensor_value_to_double()` converts this into a normal `double` for easy printing.

## 3.7 Build, flash, and observe

1. **Pristine build** — a clean build ensures your new `prj.conf` and `app.overlay` are picked up correctly:
    - VS Code: nRF Connect extension → select your build configuration → move the mouse over **Build** to open the build actions. Select **Pristine Build** by clicking the **circular arrow icon** next to it.
2. **Flash** to the DK
3. **Open the Serial Terminal** (nRF Connect for VS Code → Connected Devices → Serial Terminal), or an external tool such as `nRF Serial Terminal` at the DK's default baud rate.

The expected output is shown in the following screenshot.



*Serial Terminal output*

### Why do the values fluctuate slightly?

Values will fluctuate slightly around the `base-temperature` you set in the overlay (`20`) — this confirms the simulator is generating live data and your application is successfully fetching and reading it every 2 seconds.

# 4. 3. Sensor - real sensor (nRF54L15 on-chip TEMP sensor)

Next steps: enable the nRF54L15's on-chip TEMP peripheral via a DeviceTree overlay and alias, update the sensor node/channel in `main.c` to bind to it instead of the simulated sensor, then rebuild and reflash — the fetch/get logic itself stays unchanged.

## 4.1 Extending the Existing Project

Each chapter builds upon the project developed in the previous chapter. All concepts, instructions, and implementation steps introduced in the current chapter should be applied directly to the existing project.

The latest version of the project is available for download here as a ZIP file.

## 4.2 Add Sensor Hardware Configuration in DeviceTree

Let's enable the TEMP node (if not already `okay` by default for your SoC/board revision) and create the alias your code will use.

**app.overlay**

```
&temp {
    status = "okay";
};

/ {
    aliases {
        die-temp0 = &temp;
    };
};
```

### nRF54L15 on-chip TEMP peripheral

The nRF54L15 contains a **TEMP** peripheral that measures the die temperature of the chip itself (not ambient room temperature — expect it to run a few degrees above ambient once the CPU is active). Per Nordic's product specification:

- Resolution: **0.25 °C**.
- The TEMP analog electronics power down automatically after a measurement completes (low power).
- To hit the accuracy stated in the electrical specification, the **crystal oscillator (HFXO) must be the HFCLK source** during measurement — this is already the default configuration on the nRF54L15, so you don't need to change anything for this workshop.

## 4.3 Update sensor binding

In the previous chapter, we created a link from the simulated sensor definition in the DeviceTree file to the C code using `DT_NODELABEL()`. We need to update this link so that it points to the TEMP sensor. Instead of

```
#define SENSOR_NODE DT_NODELABEL(simulated_sensor)
```

we now use

main.c

```
/* Resolves to the DeviceTree node aliased as "die-temp0" in our overlay */
#define SENSOR_NODE DT_ALIAS(die_temp0)
```

## 4.4 Update reading from TEMP device

The simulated sensor uses the following line to read the temperature sensor value:

`if (sensor_channel_get(dev, SENSOR_CHAN_AMBIENT_TEMP, &temp) < 0) {`

The **TEMP device** uses a different sensor channel for reading the temperature. Therefore, instead of `SENSOR_CHAN_AMBIENT_TEMP`, the **die temperature** channel must be used:

`SENSOR_CHAN_DIE_TEMP`

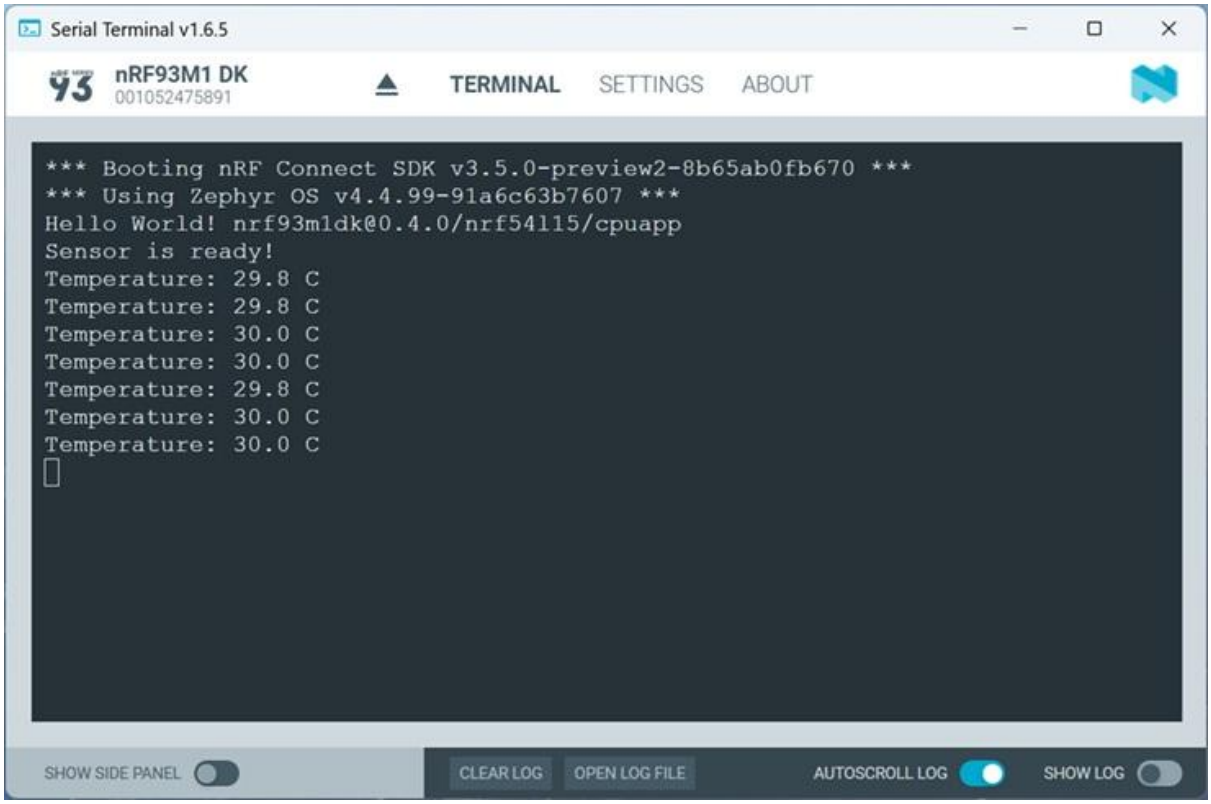
The resulting code is:

```
if (sensor_channel_get(dev, SENSOR_CHAN_DIE_TEMP, &temp) < 0) {
```

## 4.5 Build, flash, and observe

1. **Pristine build** — a clean build ensures your new `prj.conf` and `app.overlay` are picked up correctly:
    - VS Code: nRF Connect extension → select your build configuration → move the mouse over **Build** to open the build actions. Select **Pristine Build** by clicking the **circular arrow icon** next to it. The application is then rebuilt from a clean build directory.
2. **Flash** to the DK
3. **Open the Serial Terminal** (nRF Connect for VS Code → Connected Devices → Serial Terminal), or an external tool such as `nRF Serial Terminal` at the DK's default baud rate.

Expected output:




*Serial Terminal*

# 5. 4. AI-assisted development

Nordic Semiconductor's AI-assisted development integrates AI coding capabilities with the Nordic development ecosystem through the Model Context Protocol (MCP). Within VS Code, the AI assistant can combine general software-development reasoning with Nordic-specific documentation, APIs, SDK configuration, project context, and development tools. This enables the assistant to support developers across the embedded-software lifecycle—from understanding and generating application code to configuring the nRF Connect SDK, resolving build issues, and interacting with target hardware. The MCP-based approach provides the AI with structured access to relevant development context and tools, while keeping the developer in control of reviewing, validating, and executing the resulting changes.

The workshop demonstrates the practical setup of Nordic AI-assisted development in VS Code:

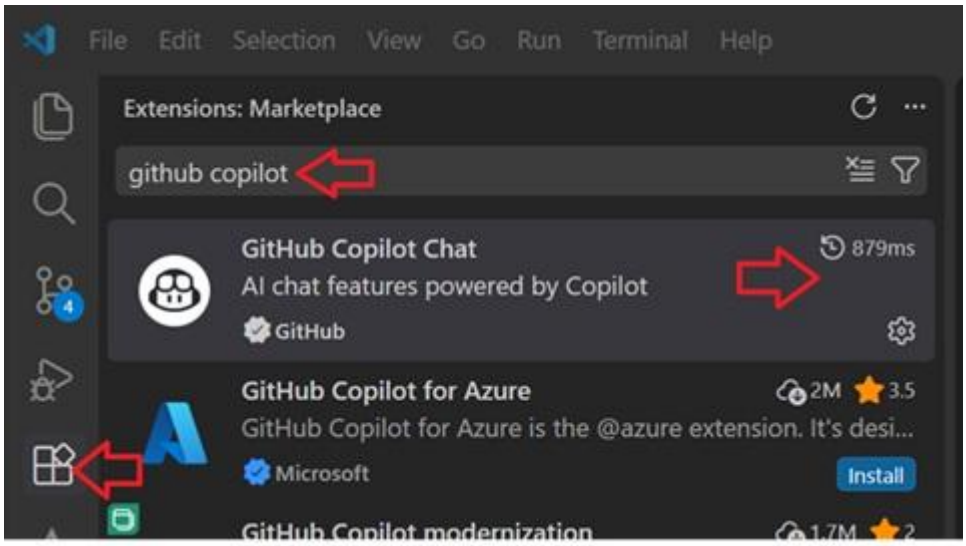
1. Start **GitHub Copilot Chat**.
2. Add and configure the **Nordic MCP server**. Connect to a **remote HTTP MCP server**.
3. Verify that the **Nordic MCP server is running and accessible**.

## 5.1 Sign up for myNordic

Create a myNordic account if you don't already have one. You will need this later to log in to the AI Assistant's user interface.

## 5.2 Github Copilot Chat

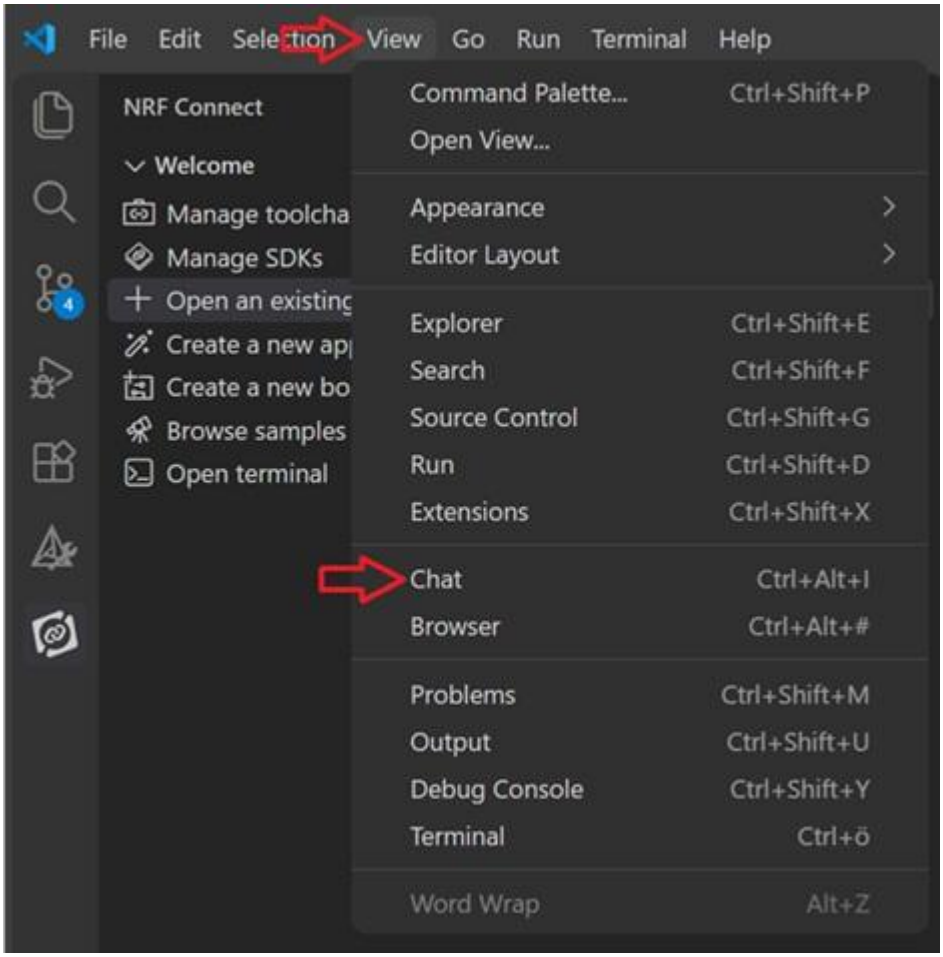
Check if Github Copilot Chat extension is installed by looking for it in Extensions




*Github Copilot Chat in Extensions*

## 5.3 Open Chat window

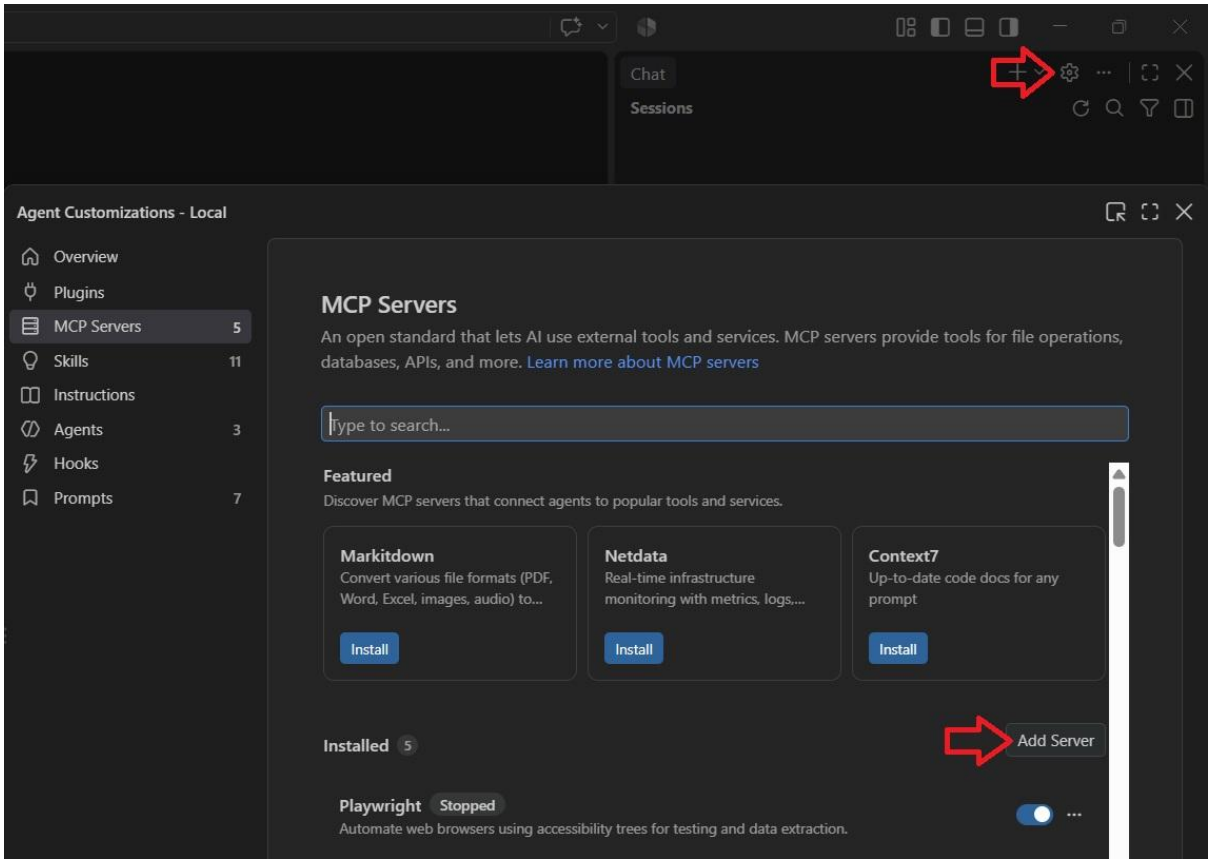
Github Copilot Chat window must be opened. It is possible that by default the Chat window opens. If not, go to menu "View" and click "Chat" to open it.



*Open Github Copilot Chat window*

## 5.4 Click gear icon and add Nordic MCP

- Click on top of the Chat window on the gear
- In the window that opens click on the **Add Server** button




*Add Server*

## 5.5 Connect to a remote HTTP server

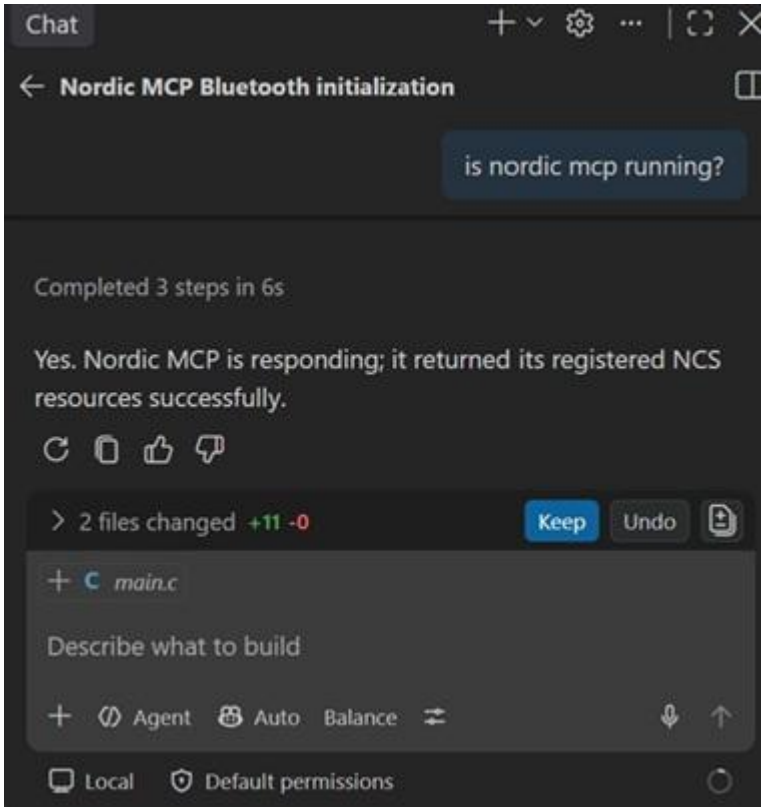
- Choose the type of MCP server to add: HTTP (HTTP or Server-Send Events)
- URL of the MCP server: **https://aidev.nordicsemi.com/mcp**
- Enter Server ID (name of the server that is used in VS Code): nordic-mcp
- Select configuration target: Global available in all workspaces, runs locally

## 5.6 Check if Nordic MCP is running

**"Is Nordic MCP running?"** is a natural way to ask whether the Nordic MCP service/process is currently up and functioning.

**Chat**

```
is nordic mcp running?
```




*Is Nordic MCP running?*

# 6. 5. Bluetooth

In this chapter, all Bluetooth stack-related functionality will be implemented using **Nordic's AI-assisted development approach with the Nordic MCP**, ensuring that the implementation follows the appropriate Nordic SDK APIs, configuration, and recommended Bluetooth architecture.

The planned Bluetooth solution will extend the existing BLE implementation without changing its established stack initialization or advertising mechanism. The solution will add **LE Secure Connections pairing** to provide a secure connection between the device and a BLE central, while intentionally **excluding bonding and persistent pairing information**. In addition, the GATT database will be extended with the standard **Battery Service (BAS)**, including a readable and notifiable Battery Level characteristic.

The application will provide an interface to update the battery level and notify connected clients when notifications are enabled. For initial testing, the battery level can be simulated using a counter that decreases from 100% to 0% and then starts again at 100%.

The connection lifecycle will also be completed by handling **BLE disconnections and automatically restarting connectable advertising**. This ensures that after a connection is lost, the device becomes discoverable again and can accept a new connection without requiring a device reset.

Overall, the solution will provide a complete BLE flow:

**Advertising → Connection → LE Secure Connections pairing → GATT/BAS access → Battery notifications → Disconnection → Advertising restart → Reconnection.**

## 6.1 Extending the Existing Project

Each chapter builds upon the project developed in the previous chapter. All concepts, instructions, and implementation steps introduced in the current chapter should be applied directly to the existing project.

The latest version of the project is available for download here as a ZIP file.

## 6.2 Use Nordic MCP: Init Bluetooth Stack and start advertising

Look for the **permissions dropdown** near the chat input. And please set the permissions to "Allow all".

Let's enter the following prompt in the VS Code Chat window, that should add Bluetooth initialization and advertising to the project.

⚠️ **Take care:** The text we copy into the chat window contains a placeholder for entering a unique name. Before you start editing in the VS Code chat window, enter your own name. See “4. Set the Bluetooth ...”

**Bluetooth stack initialization and advertising prompt**

```
Use the Nordic MCP server to add Bluetooth initialization and advertising to my existing
project.

Please:
1. Initialize the Bluetooth stack using the appropriate APIs and configuration for this
project and its target board.
2. Check and handle the initialization result before proceeding.
3. Only after successful Bluetooth initialization, start Bluetooth LE advertising.
4. Set the Bluetooth advertising/device name to <ENTER YOUR BT ADV NAME HERE>. The name
must start with BAS_.
```

```
5. Use the existing project structure and nRF Connect SDK conventions. Do not introduce
unnecessary changes to unrelated code.
6. Update any required Kconfig or project configuration so that the Bluetooth
functionality is enabled.
7. Clearly identify all files and configuration changes you make.
8. All Bluetooth functions should be placed in the file bluetooth.c. Create a header
file bluetooth.h that declares a function that declares the bluetooth_init() function..

Use the Nordic MCP documentation and tools to verify that the implementation is
appropriate for the project's nRF Connect SDK version and target board.
```

## 6.3 Test your project

- Review the proposed changes. If they are sensible and consistent with the intended behavior, choose **Keep**; otherwise choose **Undo**.
- Flash the resulting software/firmware to the dev kit.
- Open the **Serial Terminal**.
- Reset the dev kit and inspect the received logs for errors, crashes, or unexpected behavior.
- Use **nRF Connect for Mobile** to scan for BLE advertising and verify that the expected device/advertisement is visible.

## 6.4 Add connection handling and BAS profile

Add connection establishment and BAS GATT service to the existing Bluetooth implementation.

**Bluetooth Connection + BAS Prompt**

```
Use Nordic MCP.
Extend the existing Bluetooth LE implementation to include connection establishment and
the Battery Service (BAS).
The Bluetooth stack is already initialized and connectable advertising is already
running. Do not reinitialize the Bluetooth stack, recreate advertising, or replace
existing Bluetooth functionality.
First inspect the existing project to determine the Bluetooth SDK/framework and follow
its existing APIs and architecture.

Requirements:
- Add Bluetooth connection establishment.
- Handle connection success and failure through the existing Bluetooth event/callback
mechanism.
- Add the standard Bluetooth SIG Battery Service (BAS) to the existing GATT database.
- Add the Battery Level characteristic with read and notification support.
- Provide an application API to update the battery level (0–100%).
- In the main loop, a variable named “battery_level” should be used, whose value is
decremented from 100 to 0 and which is used for BAS notifications.
- Send Battery Level notifications when the connected client has enabled them.
- Preserve the existing initialization, advertising, connection, and application
architecture.
- Do not invent APIs; use APIs available in the detected SDK/version.
- Build the project after implementation and fix compilation errors and warnings
introduced by the changes.
- Add all Bluetooth code to the bluetooth.c file.
- Handle a disconnect and restarting of advertising.

After implementation, briefly summarize:
- Modified files.
- Connection establishment behavior.
- BAS implementation.
- Battery Level update API.
```

```
- Build/validation results.
```

## 6.5 Test your project

- Review the proposed changes. If they are sensible and consistent with the intended behavior, choose **Keep**; otherwise choose **Undo**.
- Flash the resulting software/firmware to the dev kit.
- Open the **Serial Terminal**.
- Reset the dev kit and check the received logs for expected startup messages, errors, warnings, crashes, or unexpected behavior.
- Use **nRF Connect for Mobile** to scan for BLE advertising and verify that the expected device/advertisement is visible.
- Check advertising restart after disconnect.
- Check BAS (Battery Service):
  > Verify that BAS service is present after connecting.
  > Read current battery level.
  > Enable Battery Level notifications.
  > Verify that notifications can be enabled successfully.

# 7. 6. PMIC

## 7.1 Extending the Existing Project

Each chapter builds upon the project developed in the previous chapter. All concepts, instructions, and implementation steps introduced in the current chapter should be applied directly to the existing project.

The latest version of the project is available for download here as a ZIP file.

## 7.2 Include nPM13xx and Fuel Gauge software library

Now add the libraries and drivers required for the nPM13xx to our project.

Fuel Gauge is based on Zephyr's sensing subsystem. This means we need to set `CONFIG_SENSOR=y`. We already did this in an earlier step when we integrated the simulated sensor. For this reason, it won't be set again here. However, the other KCONFIG settings for nPM1300 still need to be set in prj.conf.

**prj.conf**

```
# Enable I2C support
CONFIG_I2C=y

# Enable float-based math
CONFIG_FPU=y

# nPM13xx Fuel Gauge support
CONFIG_NRF_FUEL_GAUGE=y
```

## 7.3 Get a battery model

- **Locate the existing sample's model file** in your NCS checkout. `nrf/samples/pmic/native/npm13xx_fuel_gauge/src/battery_model.inc`
- **Copy that file into your project's `src/` folder**, unmodified, so you're using a real, previously-generated model rather than typing numbers by hand.
- **Include the battery model by adding below code lines**

**main.c**

```
static const struct battery_model battery_model = {
#include "battery_model.inc"
};
```

### What would I do with a battery I'd chosen myself?

If no predefined battery model matches, then you would do the following.

Install nRF Connect for Desktop's nPM PowerUP app, connect the nPM1300 EK (+ nPM Fuel Gauge Board for a rechargeable model), and profile the actual battery you're using. PowerUP exports the same kind of `.inc` file — swap it in for a model that matches your specific cells.

## 7.4 Update nPM13xx binding

**Get the device handle and check readiness first**, print-and-flash before doing anything else.

main.c

```
#include <zephyr/device.h>
#include <zephyr/devicetree.h>

static const struct device *charger = DEVICE_DT_GET(DT_NODELABEL(npm1300_charger));
```

main.c => main() function

```
    if (!device_is_ready(charger)) {
        printk("Charger device not ready\n");
        return 0;
    }
    printk("Charger device OK\n");
```

## 7.5 Add the sensor read function

**Add the sensor read function** and print raw values, before touching the fuel gauge library at all — this isolates "can I talk to the PMIC" from "does the algorithm work".

**NOTE:** The measured values are also read in here using the Zephyr Sensing subsystem. This means we also use the functions `sensor_sample_fetch()` and `sensor_channel_get()` here.

main.c => Add to the include list at the beginning of the file

```
#include <nrf_fuel_gauge.h>
```

main.c

```
static int read_sensors(float *voltage, float *current, float *temp)
{
    struct sensor_value value;
    int err = sensor_sample_fetch(charger);
    if (err) {
        printk("Error: nPM13xx sample update error\n");
        return err;
    }

    sensor_channel_get(charger, SENSOR_CHAN_GAUGE_VOLTAGE, &value);
    *voltage = (float)value.val1 + ((float)value.val2 / 1000000.f);

    sensor_channel_get(charger, SENSOR_CHAN_GAUGE_AVG_CURRENT, &value);
    *current = (float)value.val1 + ((float)value.val2 / 1000000.f);

    sensor_channel_get(charger, SENSOR_CHAN_GAUGE_TEMP, &value);
    *temp = (float)value.val1 + ((float)value.val2 / 1000000.f);

    return 0;
}
```

## 7.6 Fuel Gauge library

Bring in the fuel gauge library and initialize it.

**Note:** The code snippet assumes that the variable `err` was already declared in the previous step. Since the code generated in previous chapter varies, it is possible that a different variable name was used during the variable declaration or the error handling is done within the *bluetooth.c* file. In that case, you must declare the variable `err` here:

`int err;`

**main.c => main() function**

```
    float batt_voltage, batt_current, batt_temp, batt_soc;

    /* First reading should be taken with the battery at rest for an
     * accurate baseline (see Nordic's fuel gauge app note). */
    read_sensors(&batt_voltage, &batt_current, &batt_temp);

    struct nrf_fuel_gauge_init_parameters params =
           NRF_FUEL_GAUGE_DEFAULT_INIT_PARAMETERS_SECONDARY(batt_voltage,
                                                            batt_current,
                                                            batt_temp,
                                                            &battery_model);
    err = nrf_fuel_gauge_init(&params, NULL);
    if (err) {
        printk("Fuel gauge init failed: %d\n", err);
        return 0;
    }
    printk("Fuel gauge initialized\n");
```

## 7.7 Polling loop

Add the polling loop last.

**Note:** place below code in main() function before the

`bluetooth_battery_level_update()`

function is called. This allows us easily to replace the simulated `batt_level` by the real State-of-Charge (`soc`) value.

**main.c => main() function => in entire loop**

```
        // Fuel Gauge: State-of-Charge calculation
        read_sensors(&batt_voltage, &batt_current, &batt_temp);
        err = nrf_fuel_gauge_process(batt_voltage, batt_current, batt_temp, 1.0f,
&batt_soc, NULL);
        if (err) {
            printk("Fuel gauge process error: %d\n", err);
            continue;
        }
        printk("V: %.3f, I: %.3f, T: %.2f, SoC: %.2f%%\n",
               (double)batt_voltage, (double)batt_current, (double)batt_temp,
(double)batt_soc);
```

## 7.8 Use State-of-Charge value for Bluetooth Notification

In the previous chapter, we transmitted the battery level via Bluetooth. The level was simulated in that case. Since we now have a calculated SOC level, we can use it.

We will modify the function

`err = app_battery_level_update(batt_level);`

as shown below. Note that the `batt_level` simulation can be removed.

**Note**: It's possible that the AI tool used a different function name in your project.

**main.c => use SoC from Fuel Gauge instead of simulated value**

```
        err = app_battery_level_update(batt_soc);
```

## 7.9 Disable Charger

Since we are using the default settings of the nRF93M1-DK and have not configured the nPM1300 charger for our battery, we are disabling the charger functionality.

The battery voltage, current, and temperature are still being measured and used to calculate the battery level using the fuel gauge libraries.

Please note that after downloading the software to the hardware, you must power-cycle the hardware.

**Notes:**

- In the DeviceTree, properties of type boolean are disabled by deleting the property.

- The configuration for a selected battery is done using the npm Power Up tool.

`app.overlay`

```
&npm1300_charger {
    /delete-property/ charging-enable;
};
```

## 7.10 Build, flash, and test

- Perform a build. (pristine build is not needed this time!)
- After the build is successful, we'll flash the software to the dev kit.
- Use “NRF Connect for Mobile” to connect to your device. Enable BAS notifications and check whether the SOC value in the Serial Terminal matches the battery level in the app.

# 8. 7. Cellular

The Cellular chapter is a hands-on guide to the nRF93M1 Cat 1 bis modem, controlled entirely through AT commands. Participants set up the DK, subscribe to the modem's unsolicited notifications, connect to the cellular network, and upgrade the modem firmware from 1.4.1 to 1.5.0 over the air. They then onboard the device to nRF Cloud and use its services: location (cell and Wi-Fi based), messaging over CoAP, and device observability with metrics uploads.

## 8a. Getting Started with the nRF93M1-DK

In this chapter, we will work hands-on with the **nRF93M1 Cat 1 bis modem**. Starting with basic AT commands, we will connect the modem to a cellular network and gradually move on to more advanced features. These include performing an **OTA firmware update**, provisioning the device to nRF Cloud, determining its location using n**RF Cloud services**, and exchanging messages using **CoAP**.

### 8a.1 Connect the nRF93M1-DK

Connect the nRF93M1 DK to your computer using USB1. This USB connection provides power to the DK and connects to the nRF5340 running the SEGGER J-Link OB firmware.

USB2 is connected directly to the nRF93M1 USB interface.

The VCOM1 connection can be switched between UART0 of the nRF54L15 and UART1 of the nRF93M1. The routing is controlled by P2.06 of the nRF54L15.

With the Modem bypass application loaded, the J-Link interface exposes two virtual COM ports:

| Virtual com-port | Device | Description |
|---|---|---|
| VCOM0 | nRF54L15 | Connected to the nRF54L15 UART1 - Zephyr Shell |
| VCOM1 | nRF93M1 | Connected to the nRF54L15 UART0 or nRF93M1 DK MAIN UART1 |

To communicate directly with the nRF93M1 AT command host interface, first identify the virtual COM ports assigned to the DK. From the command line, run:

```
nrfutil device list
```

**Expected output**

```
>nrfutil device list
1052475354
Product         J-Link
Board version   PCA10232
Ports           COM110, vcom: 0
                COM111, vcom: 1
Traits          devkit, jlink, seggerUsb, serialPorts, usb

Found 1 supported device
```

### 8a.2 Restore the nRF54L15 firmware on the nRF93M1-DK

The **Modem bypass** sample is preprogrammed on the nRF93M1 DK at the factory. In the previous steps, we replaced the default firmware running on the nRF54L15. To communicate with the nRF93M1 modem through a serial terminal, we need to restore this firmware.

The default application is the **Modem bypass** sample from *NCS v3.5.0-preview2*:

`nrf/samples/nrf93m1dk/modem_bypass`

Although you can build the sample yourself, a precompiled Intel HEX file is provided for convenience:

modem_bypass.hex => right-click this link and in context menu select "Save link as..."

Use the **Programmer** application in **nRF Connect for Desktop** to flash the HEX file to the nRF54L15.

Alternatively, the firmware can be flashed from the command line using nRF Util

**Restore nRF54L15 default firmware using nRFUtil**

```
nrfutil device erase
nrfutil device recover
nrfutil device program --firmware firmware\modem_bypass.hex
```

### 8a.3 Insert SIM Card and attach antenna

The **nRF93M1 DK** comes with an **Onomondo SIM card** that includes 10 MB of data. Registering the SIM card with Onomondo provides an additional 40 MB of data.

Remove the **nano (4FF) SIM** from the SIM card and insert it into the SIM card holder on the nRF93M1 DK.

Before continuing, make sure the provided **cellular antenna** is connected to the DK.

### 8a.4 Check if nRF93M1 is responding correctly

Connect the Serial Terminal from the **nRF Connect for Desktop** application, or use any other Serial Terminal, and connect to the `VCOM1` port at 115200 baud.

Use `AT` to confirm the nRF93M1 is responding correctly.

**Serial Terminal**

```
AT
```

**Expected response**

```
> AT
OK
```

### 8a.5 Check Modem Firmware Version

Check the installed modem firmware of the nRF93M1 with: `AT+CGMR` . The nRF93M1-DKs kits are shipped with modem firmware 1.4.1 which needs to be upgrade to modem firmware version 1.5.0.

The following steps explain how to perform the modem firmware upgrade.

**Serial Terminal**

```
AT+CGMR
```

Expected response

```
>AT+CGMR
mfw_nrf93m1_1.4.1
OK
```

### 8a.6 Check Hardware Version

Check the modem hardware version using: `AT+CGMM`

This command reports the modem **hardware model/version**.

Serial Terminal

```
AT+CGMM
```

Expected Response

```
> AT+CGMM
nRF93M1-LABA-A0A (R3)
OK
```

## 8b. Setup Unsolicited Response

An unsolicited notification (formally an Unsolicited Result Code, or URC) breaks the normal AT command pattern. Normally the exchange is strictly request-response: the host sends a command, reads back the result, and the transaction is finished.

A URC is a line that arrives on the same link without any command having been sent. It is pushed asynchronously whenever some event occurs on the other side — a state change, an incoming message, a completed background operation — so its timing is not under the host's control.

We set up some unsolicited responses; use these commands before starting the connection. This will help you view the state of the connection.

### 8b.1 +CEREG: EPS network registration status

Subscribes to unsolicited EPS network registration status notifications.

Use `AT+CEREG=5` , where `5` is the URC subscription level (the maximum)

This command enables `+CEREG:` unsolicited notifications. Typically you will first see `+CEREG: 2` , which means it found a network and is trying to attach. If attaching to this cell fails, you may see another `+CEREG: 2` with a different cell.

When successfully connected, you will see either `+CEREG: 1` (registered to the home network) or `+CEREG: 5` (registered and roaming).

With this command we only enabled the notifications, it will start reporting after enabling cellular functionality with the +CFUN command in the next steps.

Serial Terminal

```
AT+CEREG=5
```

Expected Response

```
>AT+CEREG=5
OK
```

**Expected Response after enabling cellular functionality (+CFUN)**

```
+CEREG: 2

+CEREG: 5,"003B","005BC435",7
```

### 8b.2 +CGEREP: Packet domain event reporting

Enables unsolicited notifications indicating EPS Packet Data Network (PDN) connection and bearer resource status. Notifications are sent when a packet data context is activated, deactivated, or modified. This enables unsolicited messages (URCs) from `+CGEV`.

Use `AT+CGEREP=1,0`

**mode = 1**: Enable +CGEV event reporting. Events are forwarded directly to the terminal when the AT interface is available. If the interface is reserved, for example during an online data session, events are discarded rather than buffered.

After enabling it, you may see unsolicited notifications such as: `+CGEV: NW DETACH`

**Serial Terminal**

```
 AT+CGEREP=1,0
```

**Expected output**

```
> AT+CGEREP=1,0
OK
```

### 8b.3 +CSCON: Signaling connection status notification

Use `AT+CSCON=1` to enables unsolicited notifications (`+CSCON : <state>`

) this reports changes in the **RRC (Radio Resource Control)** signaling connection state.

It tells the modem to notify the host whenever it transitions between idle and connected mode. In idle mode the modem can be in the following states: DRX, eDRX, or PSM.

**Serial Terminal**

```
AT+CSCON=1
```

**Expected Response**

```
> AT+CSCON=1
OK
```

### 8b.4 +CMEE: Mobile termination error notification

When an error occurs after sending an AT command, the modem responds with ERROR by default instead of OK.

The AT+CMEE command enables extended error reporting using the final result code: +CME ERROR [err_nr]

Error codes are documented in the individual command descriptions. For a complete list, refer to 3GPP TS 27.007, §9.1.

Use AT+CMEE=2 , where 2 enables verbose error messages instead of error numbers.

Serial Terminal

```
AT+CMEE=2
```

Expected Response

```
>AT+CMEE=2
OK
```

## 8c. Connect to network

In the next steps we will connect to the network and evaluate the network connection. 

### 8c.1 +CFUN: Functional mode

The `AT+CFUN` command is used to set the modem's functional mode to **Minimum Functionality**, **Full Functionality**, or **Flight Mode**.

The time required to connect to a cellular network depends on several factors. When using a SIM card for the first time, it may take some time before the SIM card is activated on the network, depending on the operator and network conditions. Also, when using the module for the first time, it does not have information about the previously connected cell. Therefore, the modem may need to search across all enabled bands to find a suitable network.

The network registration process can be monitored using the `+CEREG` unsolicited notifications.

Use `AT+CFUN=1` , where 1 sets the module to full functionality.

Now wait until you see `+CEREG: 1` for registration on **home network** or `+CEREG: 5` for registration while **roaming**. Either response indicates that the modem has successfully registered with the cellular network.

During the registration process, you will most likely see other `+CEREG` notifications, such as:

- `+CEREG: 2` - Not registered, but currently searching for a network.
- `+CEREG: 3` - Network registration denied.

Use: `AT+CFUN=0` to switch the modem to Minimum Functionality and disconnect from the cellular network. Then reconnect using: `AT+CFUN=1`

Notice that the second connection is typically much faster. The modem can use information from the previous network connection instead of performing a complete network search again.

Setting the module to Minimum Functionality `AT+CFUN=0` or Flight Mode `AT+CFUN=4` can take several seconds. Always wait for the command to complete before continuing.

It is mandatory to switch the nRF93M1 to Minimum Functionality or Flight Mode before switching off the module.

Once the modem is registered with the cellular network, we can retrieve and examine the current connection information.

Serial Terminal

```
AT+CFUN=1
```

Expected output

```
> AT+CFUN=1
OK
```

```
+CEREG: 2

+CSCON: 1

+CGEV: ME PDN ACT 1,0

+CEREG: 5,"003B","005BC367",7

+CSCON: 0
```

### 8c.2 Connecting to the cellular network

The minimum unsolicited notification you should enable is the +CEREG=5. This is needed to get notified that you are connected to the network. You can check with AT+CEREG? the current connection status.

Note that you don't need to enable the notifications after disconnecting from the network, this is only needed after reboot/power cycle.

**Advised minimum commands to connect**

```
AT+CEREG=5
AT+CFUN=1
```

**Advised commands to connect**

```
AT+CEREG=5
AT+CSCON=1
AT+CGEREP=1
AT+CMEE=2
AT+CFUN=1
```

**An example of a connection setup**

```
> AT+CEREG=5
OK
> AT+CGEREP=1,0
OK
> AT+CSCON=1
OK
> AT+CMEE=2
OK
> AT+CFUN=1
OK

+CEREG: 2

+CSCON: 1

+CGEV: ME PDN ACT 1,0

+CEREG: 5,"003B","005BC367",7

+CSCON: 0
```

### 8c.3 +COPS: Mobile network (PLMN) selection

With the +COPS? command we can see to which operator we are connected.

The third parameter of the response, `20416` , is the numeric value of the mobile country code (MCC) and mobile network code (MNC) combined. In this case, MCC=204 (Netherlands) and MNC=16 (T-Mobile/Odido).

There are many online sources to resolve an MCC/MNC pair to an actual operator. You can check which network you are attached to using mcc-mnc.org.

Let's find out which network we are connected to, with AT+COPS?

**Serial Terminal**

```
AT+COPS?
```

**Expected output**

```
> AT+COPS?
+COPS: 0,2,"20416",7
OK
```

### 8c.4 +CGCONTRDP: PDP Context dynamic parameters

A PDP context (Packet Data Protocol context) is the data session configuration that lets a device exchange IP packets over a cellular network.

It is essentially a set of parameters, APN, IP type (IPv4/IPv6/both), and optionally authentication, that you define first, then activate. On activation, the network assigns an IP address and sets up the tunnel between the device and the gateway. Until a context is active, you have signal and network registration but no IP connectivity.

There is normally one default context that comes up automatically on attach, plus any additional ones you create and activate yourself.

Let's check which APN we are connected to, our IP address, with `AT+CGCONTRDP`.

**Serial Terminal**

```
AT+CGCONTRDP
```

**Example of output**

```
> AT+CGCONTRDP
+CGCONTRDP: 1,5,"onomodo","100.77.159.80.255.0.0.0",,"8.8.8.8","8.8.4.4",,,0
OK
```

## 8d. Update Modem Firmware

Modem firmware for an nRF93M1 module is built, signed, and released by Nordic Semiconductor, so you never compile it yourself. Your task is to deliver it, either over the air or locally through the host serial link. Modem firmware updates are separate from host application updates.

### 8d.1 Update the modem firmware over HTTP or HTTPS

The AT%HTTPFOTADL command downloads a firmware package from an HTTP or HTTPS server and triggers a firmware update. This command can only be used to update the modem firmware and not the host MCU.

Note that after the firmware update is completed the modem has done a reboot, therefor you have **lost the connection** and need to **reconnect** again to the cellular network.

To download and apply a modem firmware update:

```
AT%HTTPFOTADL="http://firmware.nrfcloud.com/93m1/mfw_nrf93m1_update_from_1.4.1_to_1.5.0.b
in",100
```

This should result in the following:

```
>
AT%HTTPFOTADL="http://firmware.nrfcloud.com/93m1/mfw_nrf93m1_update_from_1.4.1_to_1.5.0.b
in",100
AT%HTTPFOTADL="http://firmware.nrfcloud.com/93m1/mfw_nrf93m1_update_from_1.4.1_to_1.5.0.b
in",100
OK
%HTTPURC: "FOTA","HTTPSTART"
%HTTPURC: "FOTA","DOWNLOADING",0
%HTTPURC: "FOTA","DOWNLOADING",1
%HTTPURC: "FOTA","DOWNLOADING",2
-----
%HTTPURC: "FOTA","DOWNLOADING",98
%HTTPURC: "FOTA","DOWNLOADING",99
%HTTPURC: "FOTA","DOWNLOADING",100
%HTTPURC: "FOTA","DOWNLOADED"
%FOTA: "APPLY"
%FOTA: "UPDATING",0
%FOTA: "UPDATING",7
%FOTA: "UPDATING",14
-----
%FOTA: "UPDATING",92
%FOTA: "UPDATING",100
%FOTA: "DONE",0
RDY
```

## 8e. Getting Started with nRF Cloud on the nRF93M1

The nRF93M1 runs the nRF Cloud client inside the module, so your host only needs AT commands. The module still has to be known to the cloud before it can use that connection. In nRF Cloud you add the device with its UUID and registration JWT. Once it is onboarded, there are two main reasons to use the account.

**1. Cloud services (data and location)**

- `AT%NRFCLOUDMESSAGE` sends JSON data to your account.
- `AT%NRFCLOUDLOCATION` returns a position that nRF Cloud resolves from cell or Wi-Fi measurements, so no GNSS receiver is needed.
- Shadow (`AT%NRFCLOUDSHADOW`) and FOTA (`AT%NRFCLOUDFOTA`) use the same provisioned connection.

**2. nRF Cloud Observability**

The module integrates nRFCloud firmware observability. Coredumps and periodic metrics heartbeats are captured on the device, stored in the secure file system, and uploaded over the existing cellular connection. The account is where this data is received and analyzed. This is the practical way to diagnose intermittent field faults that do not reproduce on a bench.

- `AT%NRFCLOUDOBSHEARTBEAT` triggers a metrics heartbeat immediately. It serializes all accumulated metric values into the chunk queue and is intended for debug use.
- `AT%NRFCLOUDOBSUPLOAD` uploads all pending observability chunks over CoAP. An optional project key parameter overrides the key compiled into the firmware.

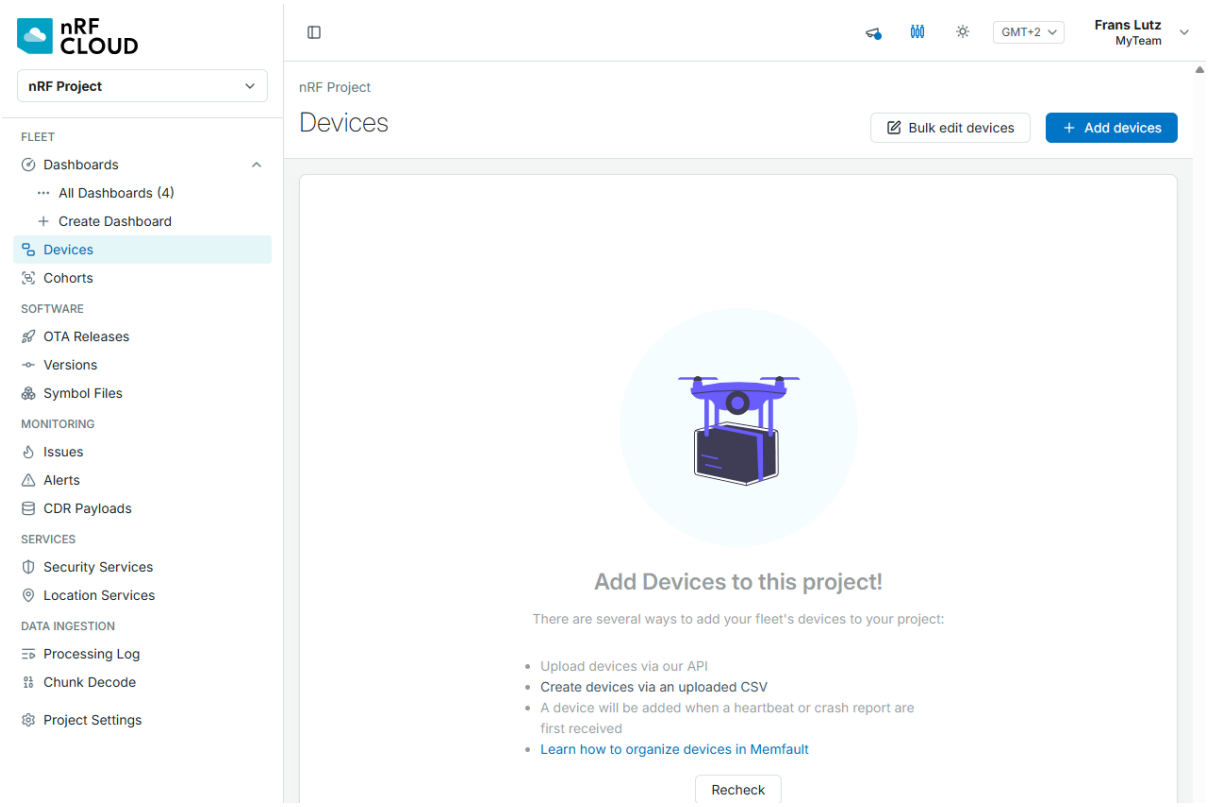
### 8e.1 Provision the nRF93M1 in nRF Cloud

Register for an nRF Cloud account, or log in to your existing account. If you already have an account, it may open the nRF Cloud legacy dashboard. Select **NEW EXPERIENCE** in the left navigation sidebar if you startup in the **Legacy Dashboard**.

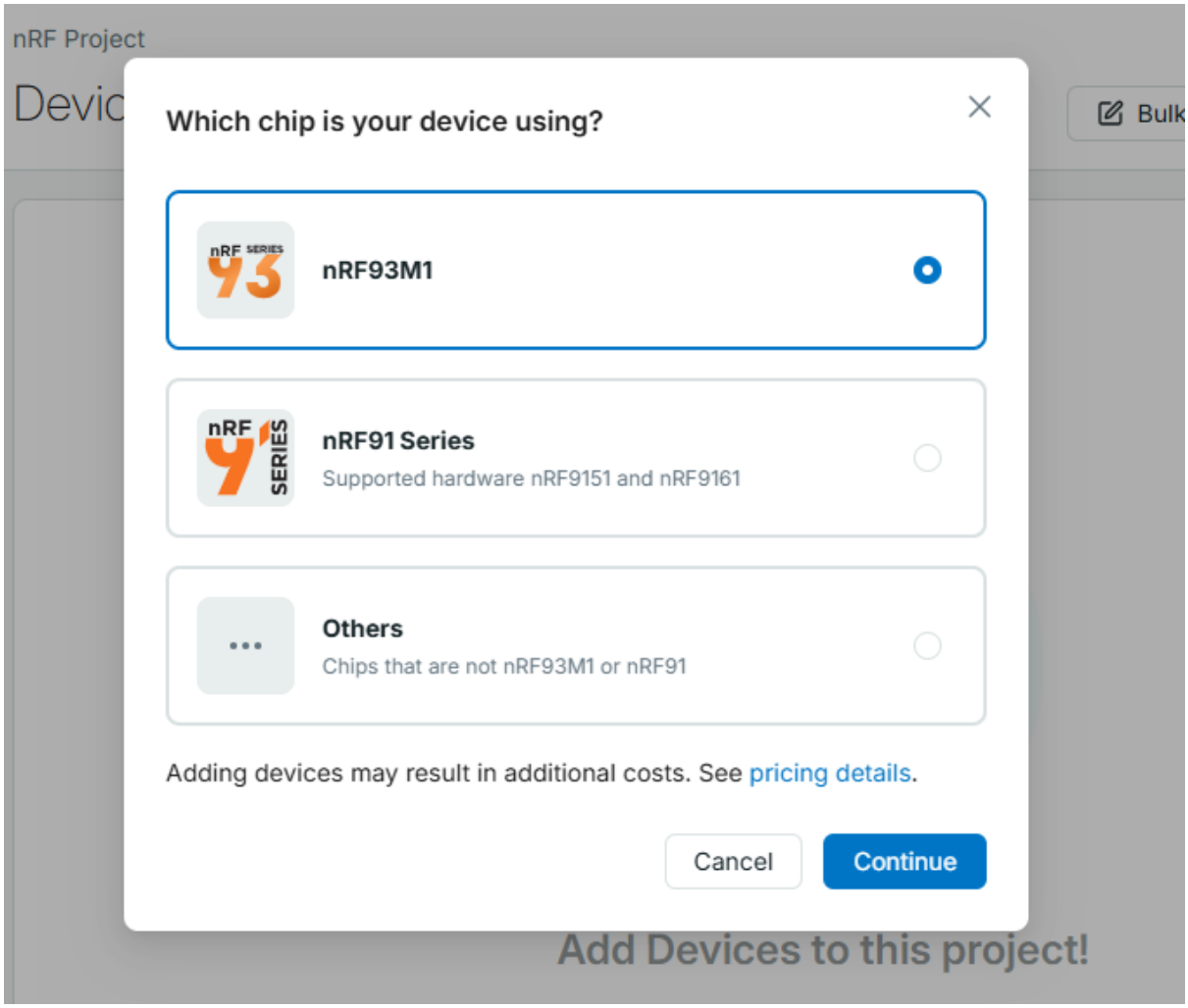
To provision the device, we need to generate a JSON Web Token (JWT) on the nRF93M1 using our **team-id** from nRF Cloud.

Navigate to **Fleet** , select **Device**, and click **Add devices**. This is the manual way to add a single device. Use **Bulk edit devices** to provision a batch of devices.

Choose the nRF93M1 from the device chooser: Select nRF93M1




*Add Devices*




*Choose the nRF93M1 from the device chooser: Select nRF93M1*

### 8e.2 Network time

Now we need to confirm the nRF93M1 has a valid time. If the device has been connected to a network since power-on, the time should already be set (some networks do not provide network time). Let's check whether the time is available by using `AT+CCLK?`

If the network didn't gave a valid time the network time will report the time the module has been powered on instead of the actual time.

A response like `+CCLK: "00/01/01,00:00:21+00"` indicates that the network time is not set.

You could connect to a network to obtain it, but during provisioning it's usually easier to set the time manually with `AT+CCLK="yyyy/MM/dd,hh:mm:ss+zz"` .

Make sure the **time is valid** before continuing to the next step. As the in the next step generate JWT token is only valid for a short period.

`Let's check whether the time has been set:`

```
AT+CCLK?
```

`Set network time:`

```
AT+CCLK="2026/10/01,09:56:00+00"
```

### 8e.3 Generate JWT Token

We now generate the JWT token on the nRF93M1 with the command `AT%REGJWT=<user-id>`

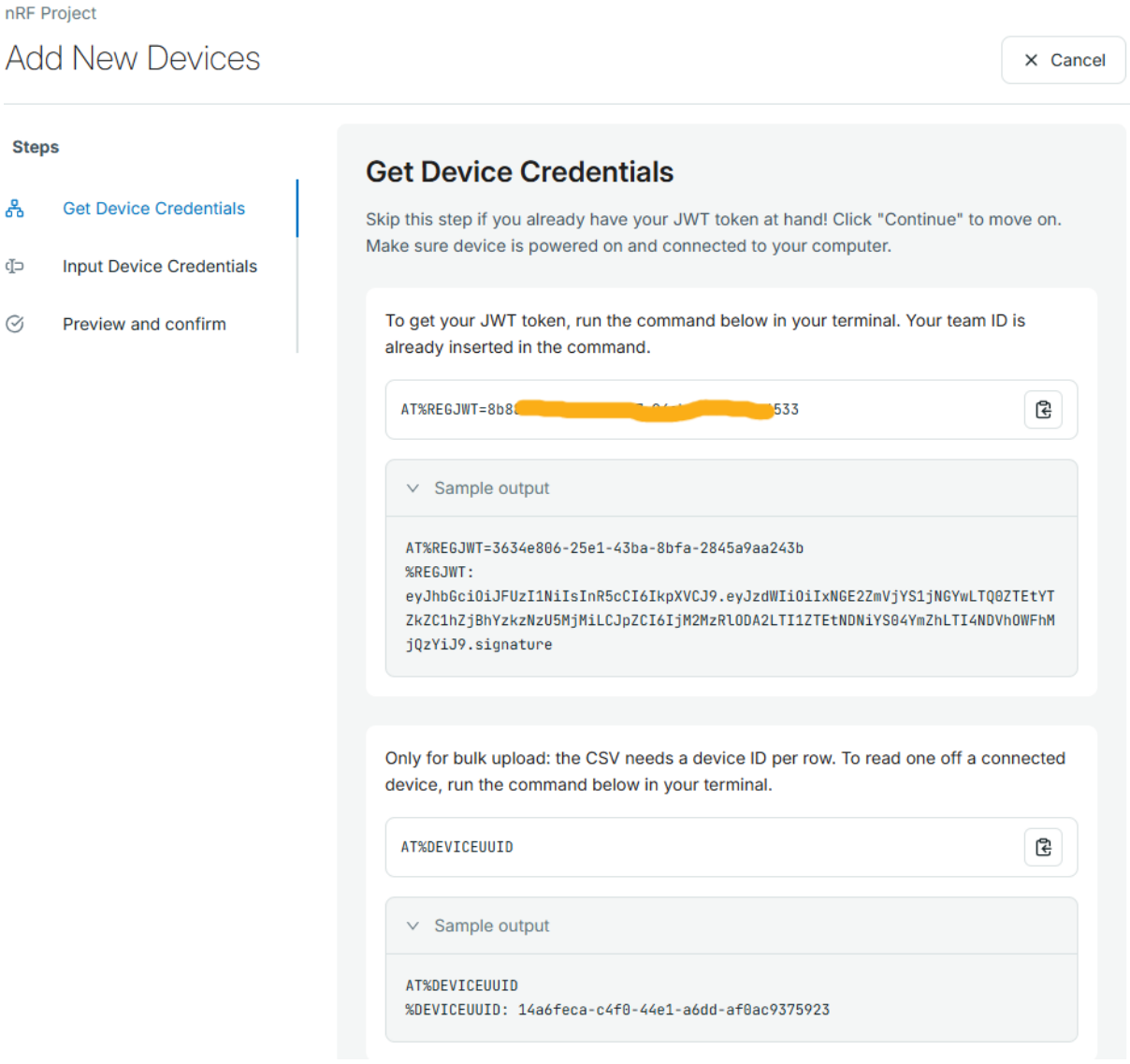
**NOTE:**

On modem firmware v1.4.1, there is a known issue where AT%REGJWT must be entered in capital letters.

In the wizard's first step, you can copy the command AT%REGJWT aith your user-id already filled in. This JWT token is only valid for one hour, which is why the nRF93M1 needs the correct time set beforehand.

Run this copied command in the terminal on the nRF93M1. After retrieving the information, press **Continue**, copy your JWT token from the serial terminal, and move to the next step.

```
Example only! Please use your own user-id.
AT%REGJWT=3634e806-xxxx-yyyy-8bfa-2845a9aa243b
```

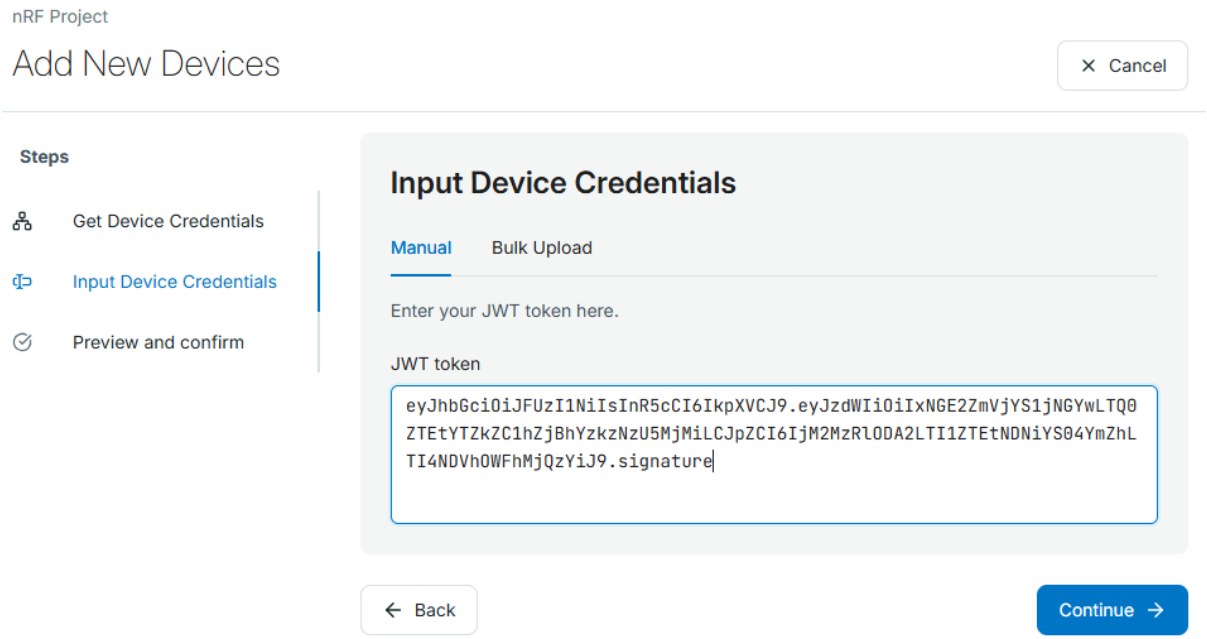



*Copy the command directly from nRF Cloud's `Add New Device` → `Get Device Credentials`, which already includes your team ID.*

### 8e.4 Input Device Credentials

Copy the JWT Token from the nRF93M1 serial terminal into the next window. Remove the `%REGJWT:` from the serial output, only copy the JWT Token.

Press **Continue** to go to the next step.

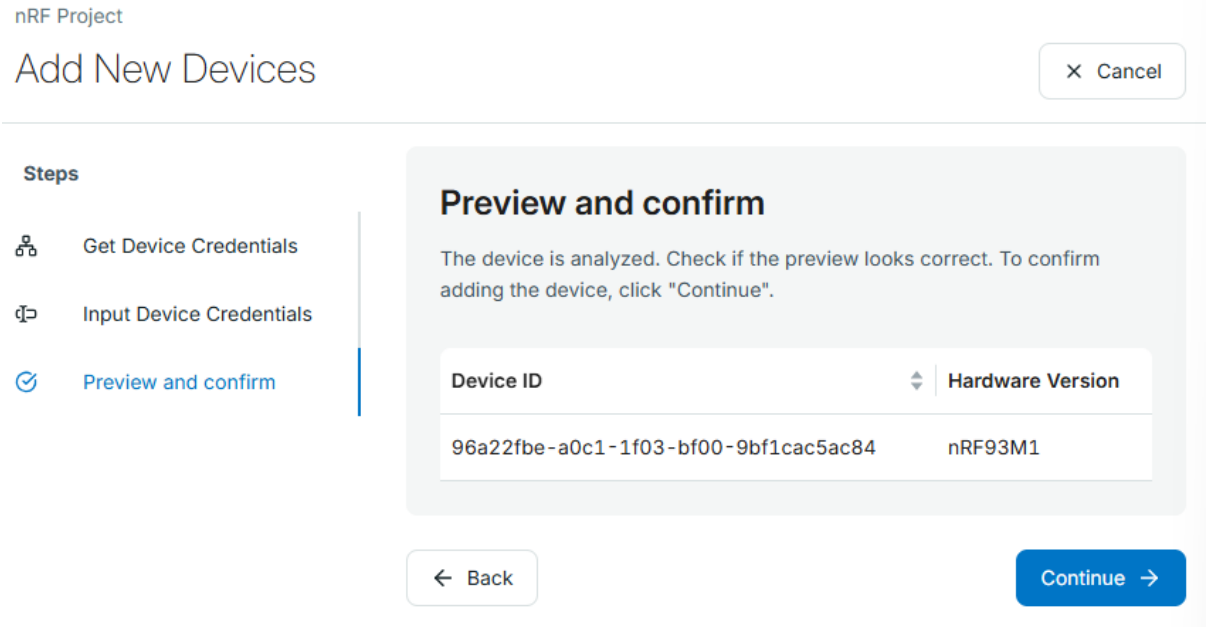



*Enter the device JWT Token*

### 8e.5 Preview and confirm

Confirm that the device UUID matches your device (`AT%DEVICEUUID`). You retrieve the UUID from the serial terminal of the nRF93M1.

Click on **Continue** to finish the provisioning.




*Confirm that UUID is correct*

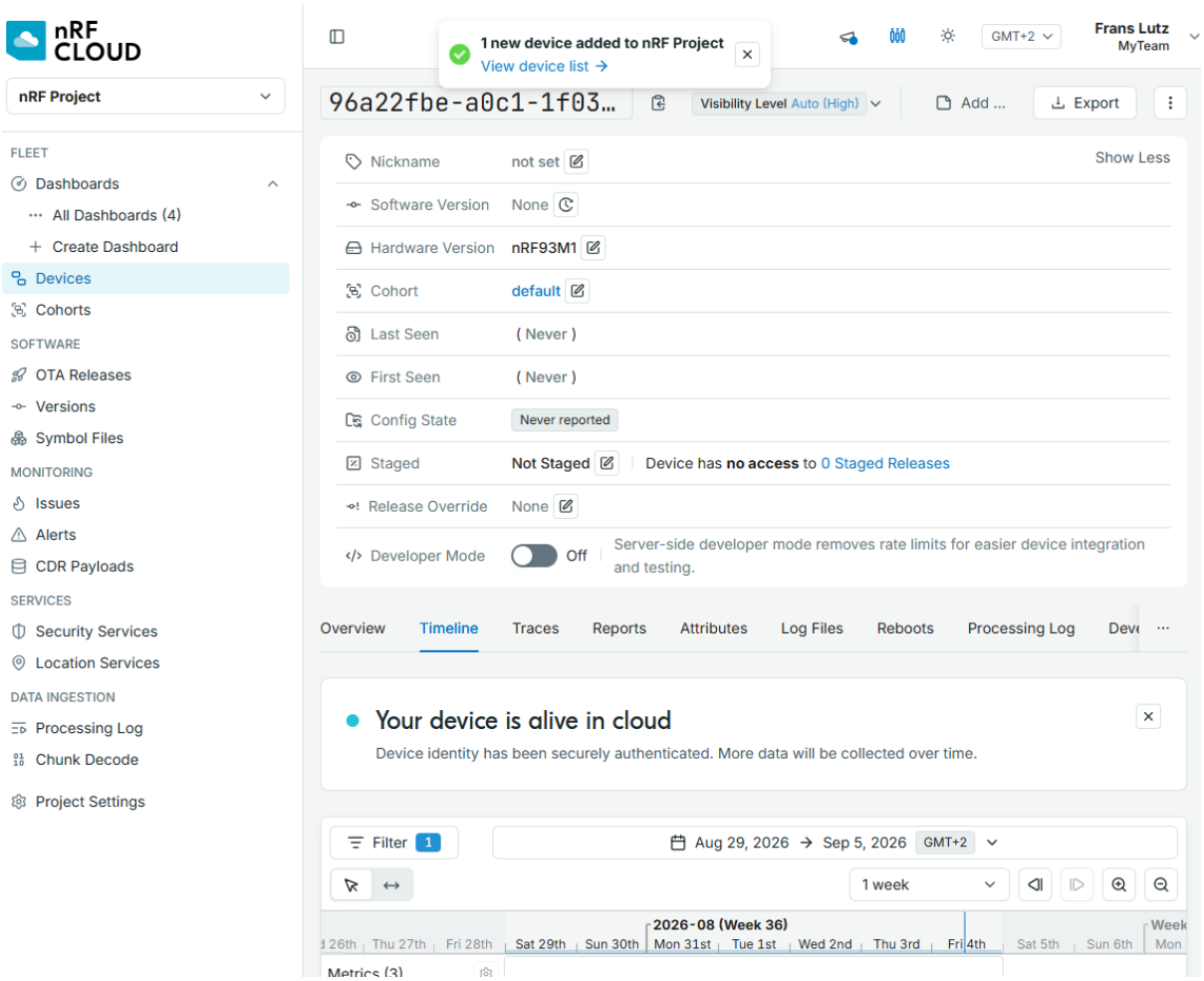
### 8e.6 nRF Cloud: Registration completed

Your device is now registered in **nRFCloud** and can be found under **Fleet -> Devices** in the left sidebar.

At this point, the device has been **provisioned**, but no communication with nRFCloud has taken place yet. Provisioning provides the device with the required credentials and authorization to connect to your nRFCloud project.

In the next steps we will use some of the `AT%NRFCLOUD` commands to communicate with nRFCloud.

**Next step: Connect to nRFCloud using AT Commands**




*Device information*

## 8f. nRF Cloud : Location Services

The nRF93M1 runs the nRF Cloud client inside the module firmware, so cloud services are reached with AT commands rather than a cloud library on the host. The transport is CoAP over DTLS 1.2, which keeps payload overhead low and coexists with cellular power saving. Every cloud transaction emits a `%COAP:` URC carrying the transport-layer status for that request, separately from the command's own response.

In the previous step, we onboarded our device to nRF Cloud. This is a mandatory procedure to follow before starting this chapter.

Some commands:

| Command | Description |
|---|---|
| %NRFCLOUDLOCATION | Cell and Wi-Fi based positioning resolved by the cloud |
| %NRFCLOUDMESSAGE | Publish a JSON message to the cloud |
| %NRFCLOUDSHADOW | Read desired shadow values, write reported shadow values |
| %NRFCLOUDFOTA | Modem firmware update and host application OTA |
| %NRFCLOUDOBSUPLOAD<br>%NRFCLOUDOBSHEARTBEAT<br>%NRFCLOUDOBSFORWARD | Memfault firmware observability |

### 8f.1 Wi-Fi Scan

`AT%WIFISCAN` uses the module's 2.4 GHz Wi-Fi beacon receiver to detect access points for location purposes only — it is a passive scan, not a Wi-Fi connection feature.

**> AT%WIFISCAN=12000,1,5,3,0**

**%WIFISCAN: (-,,-28,"00:BE:D5:20:EA:54",11)**

**%WIFISCAN: (-,,-38,"00:BE:D5:20:EA:27",3)**

**OK**

Each result line is `%WIFISCAN: (<encryption>,<SSID>,<RSSI dBm>,<BSSID>,<channel>)`. Duplicate SSIDs are removed and results are sorted by signal strength, strongest first.

The radio is shared between the LTE modem and the WiFi scan functionality. This means the modem should either be in eDRX/PSM mode or disabled. For this test disable the network connection with `AT+CFUN=0.` When the you have the result you can enable the modem with `AT+CFUN=1`.

Start the Wi-Fi scan with `AT%WIFISCAN=12000,1,5,3,0`

Description of the parameters:

| Value | Description |
|---|---|
| 12000 | Max response time in ms (5000–255000, default 12000) |
| 1 | Scan rounds (1–3, default 1) |
| 5 | Required SSID count (4–40, default 5) |
| 3 | Search time per round in seconds (1–255, default 5) |
| 0 | Priority - 0 = Data preferred / 1 = Wi-Fi preferred |

This retrieved data can be forwarded to your back office or used to obtain the latitude and longitude of your device.

**Serial Terminal**

```
AT%WIFISCAN=12000,1,5,3,0
```

**Expected response:**

```
> AT%WIFISCAN=12000,1,5,3,0
%WIFISCAN:(-,"",-83,"0A:CF:0E:18:92:33",1)
%WIFISCAN:(-,"",-83,"2E:CF:0E:21:E1:1C",1)
%WIFISCAN:(-,"petronel 9 ",-83,"5C:A6:E6:86:E7:72",2)
%WIFISCAN:(-,"",-84,"62:A6:E6:86:E7:72",2)
%WIFISCAN:(-,"",-86,"92:5C:54:CA:DF:90",6)
OK
```

### 8f.2 nRF Cloud Location services

The `%NRFCLOUDLOCATION` command triggers a cloud-based location flow.

`AT%NRFCLOUDLOCATION=<method>,<fetch_result>` collects radio measurements locally and lets nRF Cloud resolve them into a position. There are 7 different collection methods.

| Header | Header |
|---|---|
| 1 | Single-cell |
| 2 | Multi-cell |
| 3 | Single-cell and Multi-cell |
| 4 | Wi-Fi Only |
| 5 | Single-cell and Wi-Fi |
| 6 | Multi-cell and Wi-Fi |
| 7 | All methods |

The `<fetch_result>` parameter specifies whether the modem reports a location result after sending the collected data to nRF Cloud.

- 0 = Report an acknowledgment without returning a location result.
- 1 = Report the location result back to the device.

Regardless of the selected `<fetch_result>`, the location is stored on the device in nRFCloud. You can visualize this in nRF Cloud or use the result directly in Google Maps.

https://maps.google.com/?q=<lat>,<lon>

In the next step we will work with method 1, method 4 and method 5.

**Copy url and change <lat> & <lon> with the result of the NRFCLOUDLOCATION response**

```
https://maps.google.com/?q=<lat>,<lon>
```

### 8f.3 Method 1 - Single Cell

Method 1 - Single Cell: `AT%NRFCLOUDLOCATION=1,1`

Uses only the serving cell (EARFCN, PCI, cell ID, TAC, RSRP/RSRQ). Fastest and cheapest, works anywhere the module is registered, and works in RRC Connected mode. Accuracy is the size of the cell, so typically hundreds of meters to several kilometers.

**Serial Terminal**

```
AT%NRFCLOUDLOCATION=1,1
```

**Expected result**

```
> AT%NRFCLOUDLOCATION=1,1
%COAP: RESOLVING coap.nrfcloud.com (IPv4)...
%COAP: IPv4: 34.200.153.177, PORT: 5684
%COAP: CONNECTING ...
%COAP: DTLS_STATE,4
%COAP: SENDING_AUTH
%COAP: RESPONSE,2.01
%COAP: CONNECTED
%COAP: RESPONSE,2.05
%COAP:
DATA,HEX,bf01fb4049eac18cfda9e402fb40117044f8579XXXX3fb40694a2d0XXXX041904655343454c4cff
%NRFCLOUDLOCATION: 61.494029,23.775517,100,1
OK
```

#### Use the result in googlemaps

On the last line of the result : %NRFCLOUDLOCATION: 61.494029,23.775517,100,1 you find the latitude and longitude. Modify those in the URL to view the location in google maps.

`https://maps.google.com/?q=<latitude>,<longitude>`

https://maps.google.com/?q=61.494029,23.775517

### 8f.4 Method 4 - Wi-Fi only

Method 4 - Wi-Fi only: `AT%NRFCLOUDLOCATION=4,1` runs a 2.4 GHz beacon scan and sends the access point **BSSID**'s and **RSSI** values. It provides the best accuracy of the three previous cell based methods, typically tens of meters, and works indoors where cellular positioning is weakest. However, it depends entirely on access point density, a rural site may return nothing usable, and it requires RRC Idle.

Before we can send this command we first need to do disable the cellular connection and do a Wi-Fi scan. After the Wi-Fi scan we need to enable the cellular connection to get the location using nRFCloud.

**Disable cellular connection**

```
AT+CFUN=0
```

**Scan Wi-Fi access points**

```
AT%WIFISCAN=12000,1,5,3,0
```

**Enable cellular coonection**

```
AT+CFUN=1
```

**Get Location from nRF Cloud**

```
AT%NRFCLOUDLOCATION=4,1
```

**Example of the expected output**

```
> AT+CFUN=0
OK
> AT%WIFISCAN=12000,1,5,3,0
%WIFISCAN:(-,"petronel 9 ",-81,"5C:A6:E6:86:E7:72",2)
%WIFISCAN:(-,"petronel 9 ",-82,"5C:A6:E6:86:EA:F6",2)
%WIFISCAN:(-,"",-82,"62:A6:E6:86:E7:72",2)
%WIFISCAN:(-,"",-83,"62:A6:E6:86:EA:F6",2)
%WIFISCAN:(-,"Ziggo3757183",-84,"0A:CF:0E:18:92:34",1)
OK
> AT+CFUN=1
OK
+CEREG: 2
+CSCON: 1
+CEREG: 5,"003B","005BC47B",7
> AT%NRFCLOUDLOCATION=4,1
%COAP: RESOLVING coap.nrfcloud.com (IPv4)...
%COAP: IPv4: 54.145.71.75, PORT: 5684
%COAP: CONNECTING ...
%COAP: DTLS_STATE,4
%COAP: SENDING_AUTH
%COAP: RESPONSE,2.01
%COAP: CONNECTED
%COAP: RESPONSE,2.05
%COAP:
DATA,HEX,bf01fb4049ea79a430cc5102fb40116c223fc5e8ad03fb405739999999999a046457494649ff
%NRFCLOUDLOCATION: 51.831837,4.355599,92,4
OK
```

### 8f.5 Method 5 - Single-cell + Wi-Fi

Method 5 - Single-cell + Wi-Fi: `AT%NRFCLOUDLOCATION=5,1` Collects both the cell and the Wi-Fi access points and lets the cloud choose. This is the robust option for a deployed device: Wi-Fi accuracy where APs exist, automatic fallback to single-cell where they do not, instead of a failed request.

Like with the previous method (Wi-Fi only) we first need to do disable the cellular connection and do a Wi-Fi scan. After the Wi-Fi scan we need to enable the cellular connection to get the location using nRFCloud.

**Disable cellular connection**

```
AT+CFUN=0
```

**Scan Wi-Fi access points**

```
AT%WIFISCAN=12000,1,5,3,0
```

**Enable cellular coonection**

```
AT+CFUN=1
```

**Get Location from nRF Cloud**

```
AT%NRFCLOUDLOCATION=5,1
```

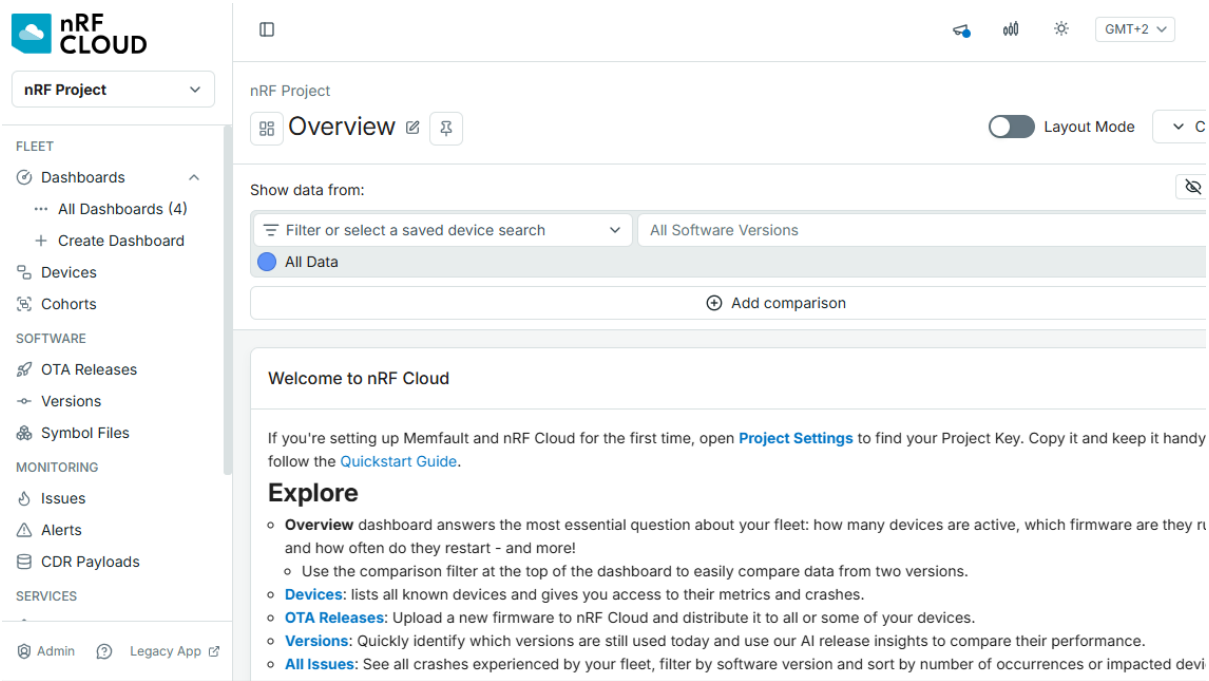
**Expected output:**

```
> AT+CFUN=0
OK
> AT%WIFISCAN=12000,1,5,3,0
%WIFISCAN:(-,"petronel 9 ",-81,"5C:A6:E6:86:E7:72",2)
%WIFISCAN:(-,"petronel 9 ",-82,"5C:A6:E6:86:EA:F6",2)
%WIFISCAN:(-,"",-82,"62:A6:E6:86:E7:72",2)
%WIFISCAN:(-,"",-83,"62:A6:E6:86:EA:F6",2)
%WIFISCAN:(-,"Ziggo3757183",-84,"0A:CF:0E:18:92:34",1)
OK
> AT+CFUN=1
OK
+CEREG: 2
+CSCON: 1
+CEREG: 5,"003B","005BC47B",7
> AT%NRFCLOUDLOCATION=5,1
%COAP: RESPONSE,2.05
%COAP:
DATA,HEX,bf01fb4049ea787c12a01602fb40116c17f2650cca03fb405899999999999a046457494649ff
%NRFCLOUDLOCATION: 51.831801,4.355560,98,4
OK
```

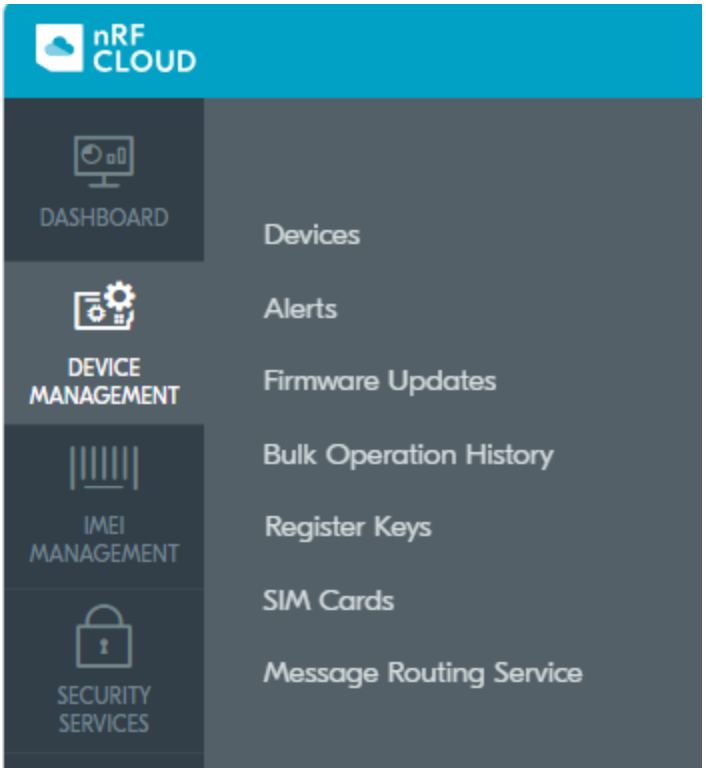
### 8f.6 Viewing the position in nRF Cloud

In nRF Cloud, we can now visualize our reported location based on the commands in the previous steps.

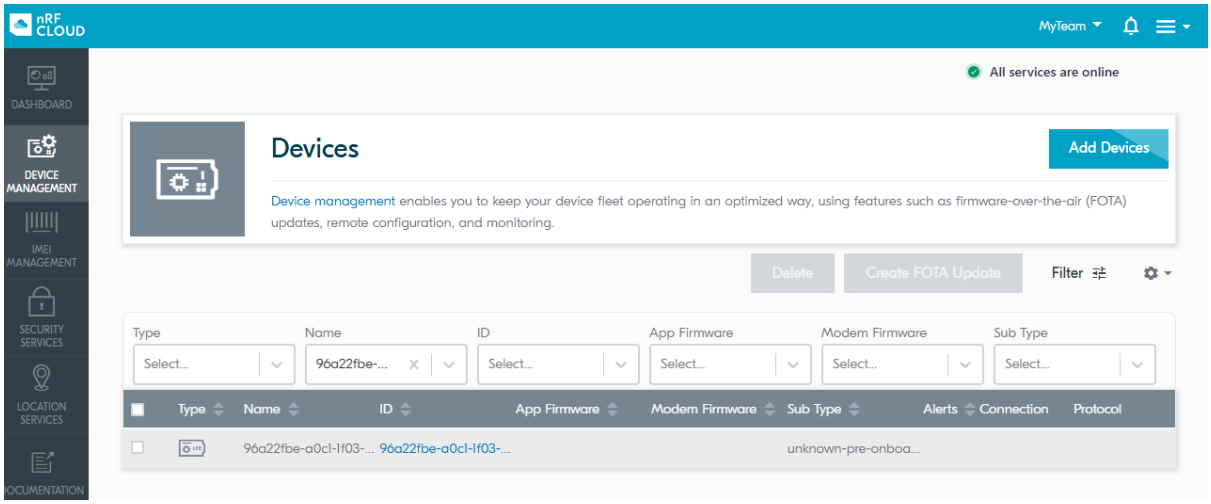
The location view is at this moment not yet transferred to the New Experience site. Go to the **Legacy App** in the left sidebar to view the live or historical location.



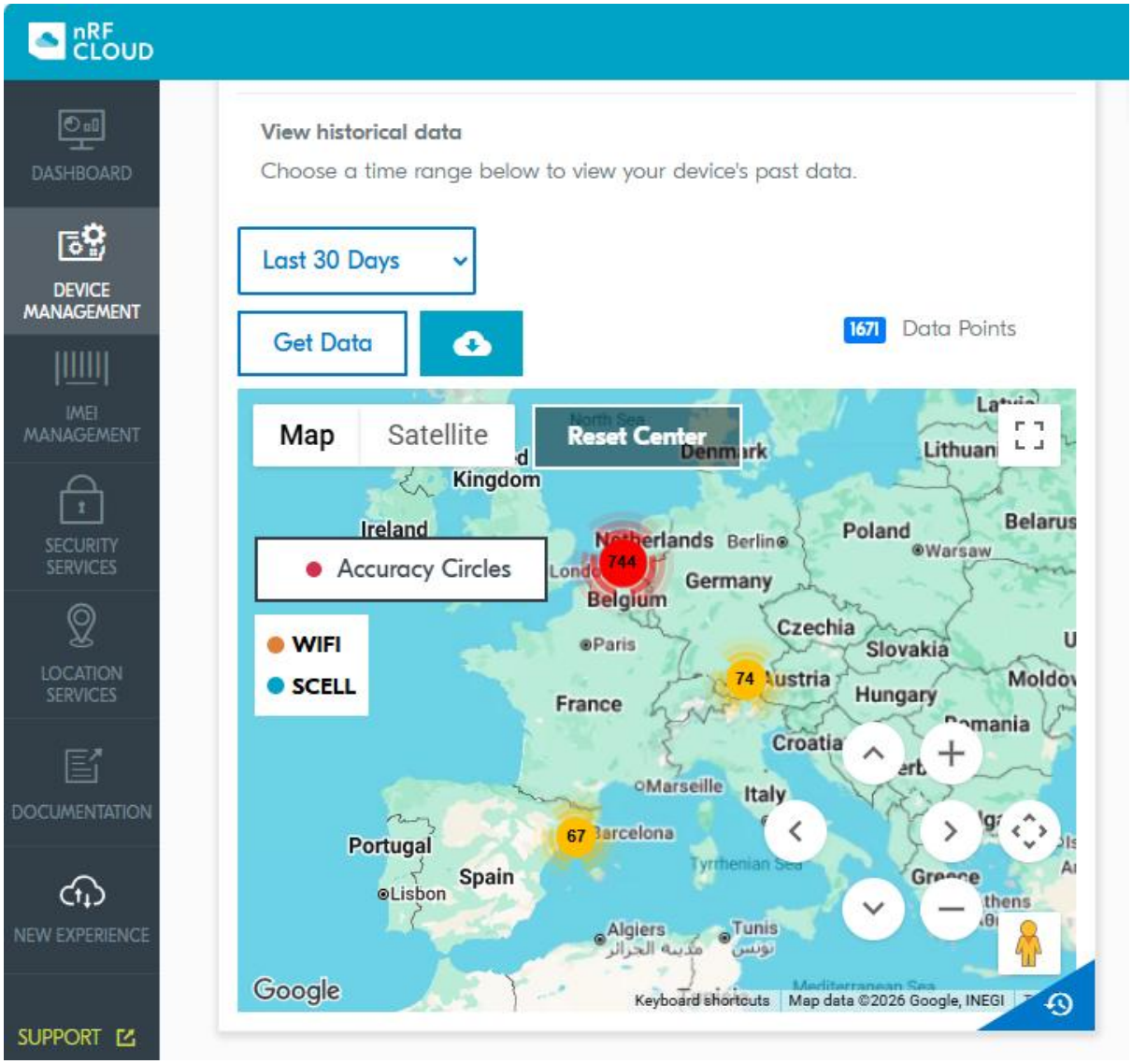

*Go to the "Legacy App" in the left sidebar.*




*Select "DEVICE MANAGEMENT" -> "DEVICES"*




*Click on your device to open a dashboard with device information.*




*View actual or historical location data*

### Location window

Scroll to the **Location** window. Here you can see the live location data or click on **View historical data**. The window shows the lookups from that device since opening this window. The blue circle represents the single-cell location, and the orange circles represent locations using Wi-Fi access points.

## 8g. nRFCloud : Messages Services

In this exercise you send data from the nRF93M1 to nRF Cloud and read it back. The module sends a short JSON message to your nRF Cloud account with AT%NRFCLOUDMESSAGE. You then retrieve the message from a terminal with a cURL request to the nRF Cloud REST API. This confirms the whole path works: from the module, over the cellular network, to the cloud, and back to your PC.

### 8g.1 nRF Cloud Messages

`%NRFCLOUDMESSAGE` is a one-way push of telemetry data.

Use: `AT%NRFCLOUDMESSAGE={"appId":"BUTTON","data":"1"}` to send a JSON object or array (max 511 characters) to nRF Cloud over CoAP. This command replies `%NRFCLOUDMESSAGE: SENT`.

Use it for events and sensor readings "fire and forget", nothing is stored on the device side.

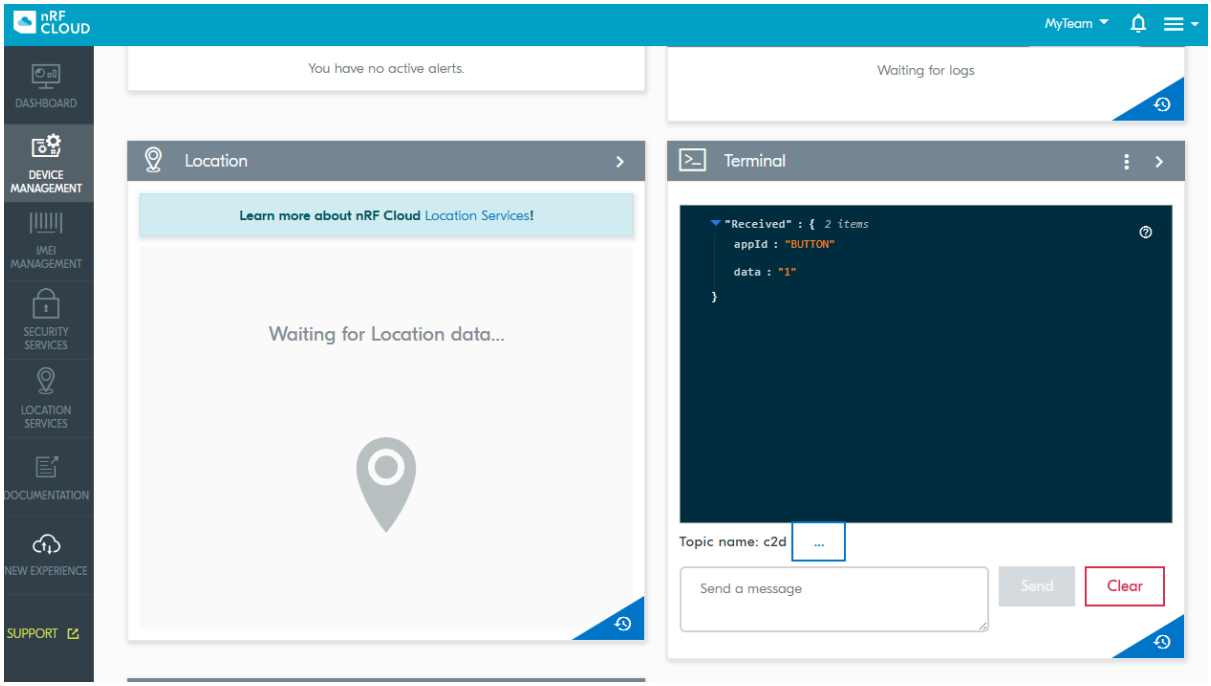
Like the location data the message can be viewed (only live data). Go to the **Legacy App** -> **Device Management** -> **Devices**. Click on your active device. Scroll to view the window "**Terminal**"

**Send some data to nRF Cloud:**

```
AT%NRFCLOUDMESSAGE={"appId":"BUTTON","data":"1"}
```

**Expected response:**

```
> AT%NRFCLOUDMESSAGE={"appId":"BUTTON","data":"1"}
%COAP: RESPONSE,2.01
%NRFCLOUDMESSAGE: SENT
OK
```




*You can view this data at the same location as the position data in nRF Cloud.*

## 8g.2 Extract Messages using REST API

You can also extract this data directly from the cloud using the REST API interface.

The endpoint is `ListMessages` : `GET https://api.nrfcloud.com/v1/messages`, authenticated with your team's API key as a simple bearer token.

The API key is found in the nRF Cloud portal under in the Legacy view. Click on the icon in the top right corner, select "**User Account**". The API Key is found in the window "**Current Team Details**".

The data can be read using for example using Postmen, for simplicity we will use the "**CURL**" command from the command prompt.

We also need the device UUID of the device we to address in nRF Cloud.

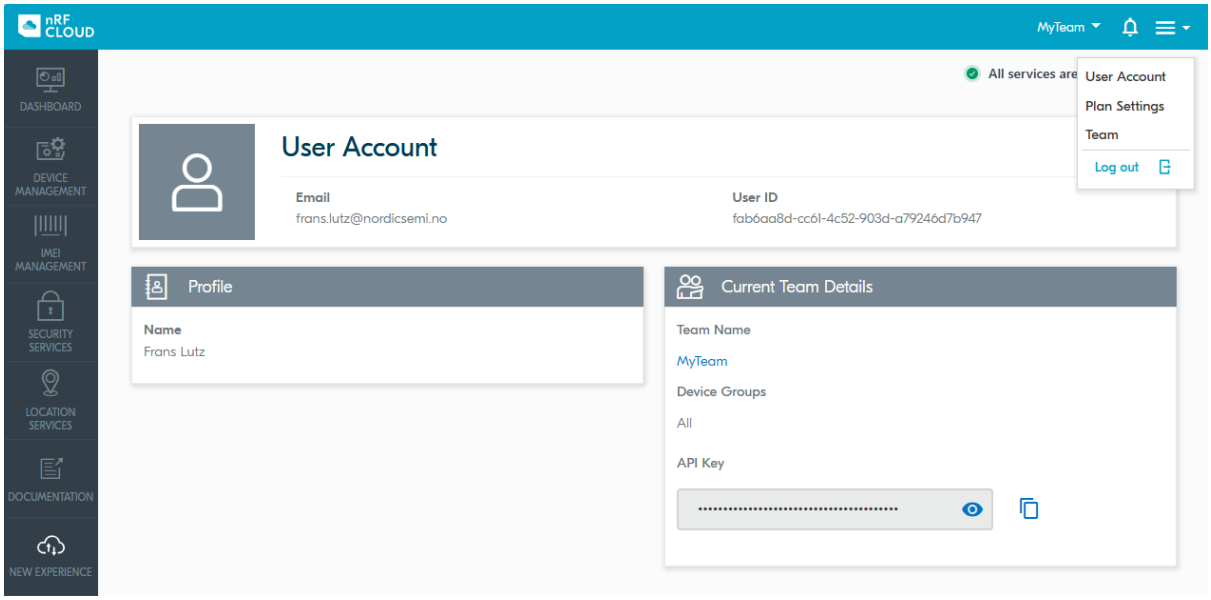
Use `AT%DEVICEUUID` to retrieve the device UUID.

Keep both the UUID and API Key we will need it in the next steps.

You find more information about the REST API interface on the nRF Cloud website.

Read UUID from nRF93M1

```
AT%DEVICEUUID
```




*Copy the API Key:*

### 8g.3 Reading device messages with the nRF Cloud REST API

The messages you send with `AT%NRFCLOUDMESSAGE` are stored in your nRF Cloud account. You can read them back from a terminal with the REST API, which is a quick way to confirm that the data really arrived in the cloud.

What the bellow command does:

- `curl -s -X GET:` sends an HTTP GET request. -s (silent) hides the progress meter so only the response is printed.
- `https://api.nrfcloud.com/v1/messages:` the endpoint that returns device messages.
- `deviceId=<YOUR_DEVICE_UUID>`: selects the device. Use the UUID you read earlier with `AT%DEVICEUUID`.
- **pageLimit=5**: returns at most 5 messages per request, which keeps the output short.
- `-H "Authorization: Bearer <YOUR_API_KEY>":` authenticates the request with your nRF Cloud API key.
- `-H "Accept: application/json":` asks for the answer in JSON format.

Steps:

1. Replace `<YOUR_DEVICE_UUID>` with the UUID of your nRF93M1.
2. Replace `<YOUR_API_KEY>` with your nRF Cloud API key.
3. Send a message from the module with `AT%NRFCLOUDMESSAGE`, then run the command.
4. Check that your message appears in the JSON response.

Tips:

- Keep the URL in quotes. Without them the shell treats & as "run in the background" and cuts the URL off.
- Add `| jq .` at the end to get a readable, colored JSON output.
- In PowerShell use curl.exe, because curl is an alias for another command.

- Treat the API key like a password. Do not paste it into chats or commit it to a repository.

**Run on command line:**

```
curl -s -X GET \
  "https://api.nrfcloud.com/v1/messages?deviceId=<YOUR_DEVICE_UUID>&pageLimit=5" \
  -H "Authorization: Bearer <YOUR_API_KEY>" \
  -H "Accept: application/json"
```

### When using windows

Copy the bellow text this doesn't have the "\" as this is not accepted in windows command prompt.

curl -s -X GET "https://api.nrfcloud.com/v1/messages?deviceId=<YOUR_DEVICE_UUID>&pageLimit=5" -H "Authorization: Bearer <YOUR_API_KEY>" -H "Accept: application/json"

## 8h. nRF Cloud : Observability Services

What is observability?

Observability means seeing what a device is doing in the field, so you can find out why something went wrong without having the device on your bench. The nRF93M1 integrates Memfault firmware observability. It records coredumps (a snapshot of the device state after a crash) and periodic metrics heartbeats, stores them in the secure file system, and uploads them over the existing cellular connection. This helps with intermittent faults that do not reproduce in the lab.

This exercise

You trigger a metrics heartbeat with AT%NRFCLOUDOBSHEARTBEAT, which collects the current metric values on the module. You then send the collected data to the cloud with AT%NRFCLOUDOBSUPLOAD, and check that it arrives in your nRF Cloud account.

### 8h.1 Set the metrics data

After successfully connecting to the network, we will create the current metrics snapshot (chunk) for later observation in nRF Cloud. Before doing this, we need to run the following commands. The results of these commands will be stored in the metrics snapshot.

Commands that are stored in the heartbeat metrics:

- `AT+CGMR` // Sets modem FW version
- `AT+CEREG=5` // Enable full CEREG URCs (PSM, AcT, etc.)
- `AT+CFUN=0` // Detach (triggers disconnect + connection_loss_count)
- `AT+CFUN=1` // Reattach (triggers time_to_connect_ms, on_time_ms)
- `AT+COPS?` // Get operator name
- `AT%BCINFO` // Gets cell information (RSRP + band)

In the previous steps, we already activated the URC for `AT+CEREG=5` and connected with `AT+CFUN=1`.

Run the following commands to update the metrics before sending them to nRF Cloud:

**Read modem firmware**

```
AT+CGMR
```

**Read network operator and country code**

```
AT+COPS?
```

**Read cell information**

```
AT%BCINFO
```

**Example response:**

```
> AT+CGMR
mfw_nrf93m1_1.5.0
OK
> AT+COPS?
+COPS: 0,2,"20416",7
OK
> AT%BCINFO
%BCINFOSC: 3300,52,-97,-9,"204","16","005BC435","003B"
%BCINFONC: 6200,52,-80,-8
%BCINFONC: 3700,52,-81,-9
%BCINFONC: 1656,52,-90,-8
%BCINFONC: 1800,52,-93,-12
%BCINFONC: 3700,163,-96,-24
OK
```

### 8h.2 Create a snapshot of the current metrics

We will use the command `AT%NRFCLOUDOBSHEARTBEAT`.

This command immediately triggers a cloud observability metrics heartbeat event and serializes all currently accumulated metric values into the chunk queue. This is not exported to the cloud, it is stored in the internal memory of the nRF93M1. This data can be uploaded to nRF Cloud using the cellular connection with an AT command or loaded into nRF Cloud using any other transport method.

**Trigger a cloud observability metrics heartbeat event**

```
AT%NRFCLOUDOBSHEARTBEAT
```

**Expected response:**

```
> AT%NRFCLOUDOBSHEARTBEAT
OK
```

### 8h.3 Upload the Modem Symbol File

The nRF93M1 sends its observability data in a compact binary format, which saves air time and power. The data contains only numbers, with no names or descriptions. The symbol file for your modem firmware version is the decode file that tells nRF Cloud what these numbers mean. With it, values such as RSRP, PSM timer, eDRX interval and modem firmware version are shown in readable form. The file must match your modem firmware version exactly, and it must be uploaded **before** the first heartbeat. Otherwise the Processing Log shows an error.

Steps:

1. Read the modem firmware version with `AT+CGMR`
2. Download the nRF93M1 decode symbol file that matches this version from the nRF93M1 LTE Metrics guide. It is also placed in the folder firmware\nrf93m1_decode_1.5.0-fa9e2900fa3e0bd04143207a07b8dad9.elf

3. Upload the file in the web app under **Software > Symbol Files > Upload Symbol File** (or use this direct link).
4. Confirm that the file appears in the list.

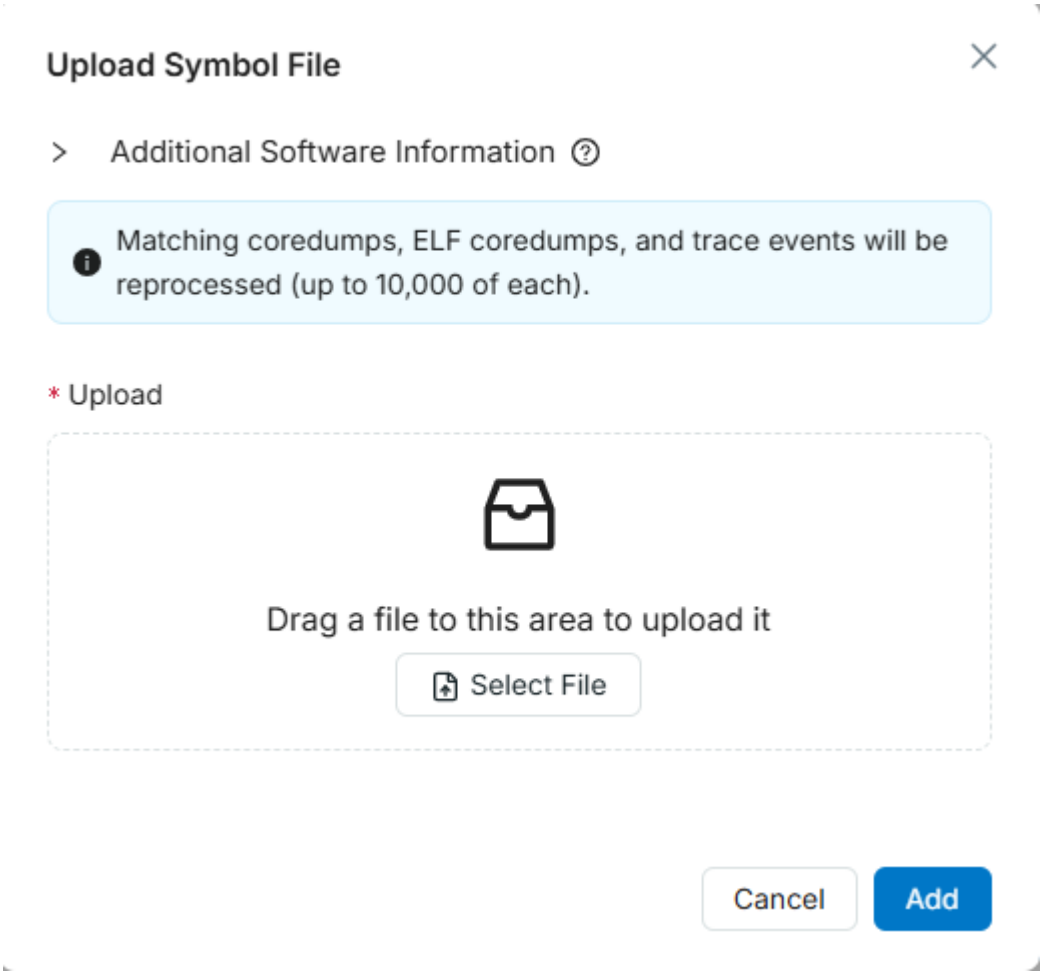
**Tip:** If you update the modem firmware later, upload the symbol file for the new version too. Data from the new version cannot be decoded with the old file.

**Read modem firmware**

```
AT+CGMR
```

**Expected Response:**

```
>AT+CGMR
mfw_nrf93m1_1.5.0
OK
```




*Upload symbol file:*

### 8h.4 Upload Observability Data to nRF Cloud

`AT%NRFCLOUDOBSUPLOAD` uploads all pending observability data (metrics heartbeats, coredumps) from the module's on-device queue to nRF Cloud over CoAP. An optional project key overrides the key compiled into the firmware.

Optionally, you can specify a **Project Key**. This is only needed when uploading to another project (a free account doesn't support multiple projects).

**Upload all pending observability data:**

```
AT%NRFCLOUDOBSUPLOAD
```

**Expected response:**

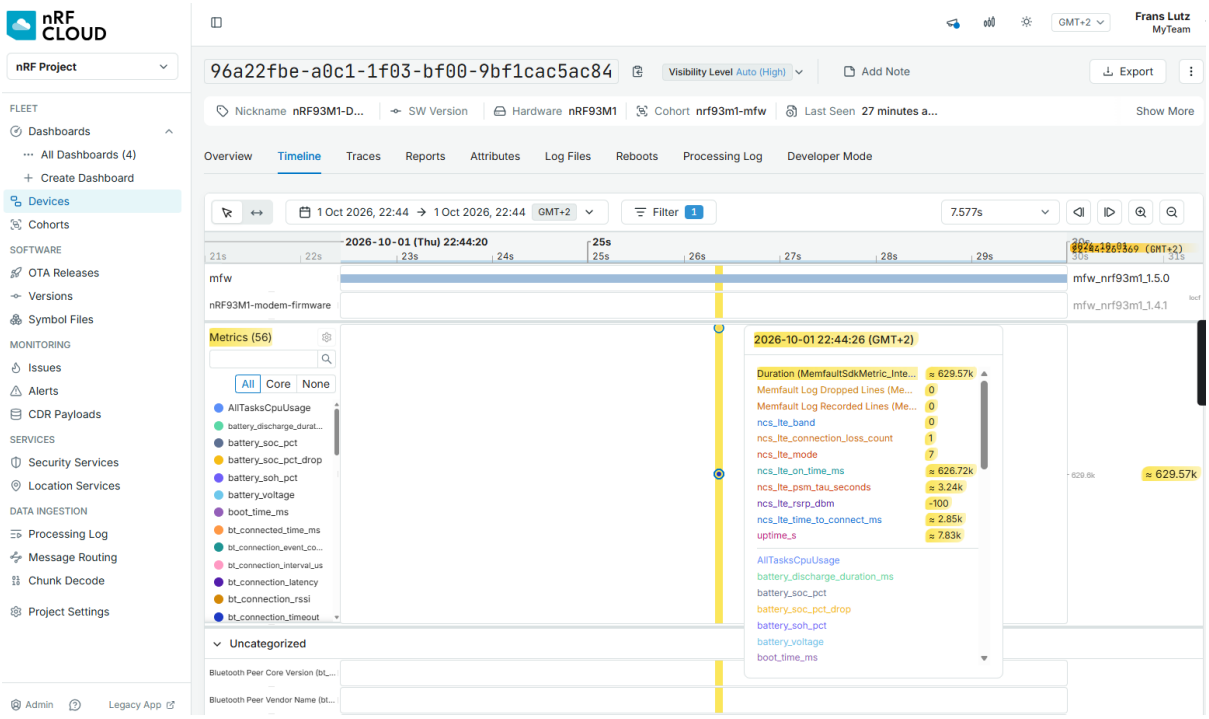
```
> AT%NRFCLOUDOBSUPLOAD
%COAP: RESOLVING coap.nrfcloud.com (IPv4)...
%COAP: IPv4: 54.145.71.75, PORT: 5684
%COAP: CONNECTING ...
%COAP: DTLS_STATE,4
%COAP: SENDING_AUTH
%COAP: RESPONSE,2.01
%COAP: CONNECTED
%COAP: RESPONSE,2.01
%COAP: DATA,8,Accepted
%COAP: RESPONSE,2.01
%COAP: DATA,8,Accepted
%COAP: RESPONSE,2.01
%COAP: DATA,8,Accepted
%NRFCLOUDOBSUPLOAD: UPLOADED,3
OK
```

## 8h.5 View the Observability in nRF Cloud

You can now view the reported data in the nRFCloud.

Select you devices in **Fleet -> Devices**, click on the tab **Timeline** and select the current date/time.

Under the tab **Processing Log** you will see the received Metric Reports.




*View the Observability in nRF Cloud*

# 9. Completed

## 9.1 Congratulation!!!

